# Proiectare Logică

I CB: Mariana Mocanu mariana.mocanu@cs.pub.ro
I CC: Anca Morar anca.morar@cs.pub.ro
**I CA & CD: Costin Chiru** costin.chiru@cs.pub.ro

Curs 7: Minimizarea funcțiilor booleene folosind diagrame Karnaugh. Circuite combinationale MSI – Curs Practic 2

# Diagrame Karnaugh – reminder

- Folosite pentru minimizare.
- Utile pentru un număr mic de variabile → pentru mai multe variabile avem Quine McKluskey.
- Două căsuțe adiacente diferă printr-un singur bit.
- Alegerea ordinii în care sunt folosite variabilele în diagramă este aleatoare, dar trebuie avut grijă cum completăm diagrama.
- Vom extinde ce ați învățat deja cu diverse cazuri particulare și exemple.

# Diagrame Karnaugh – Algoritm de minimizare (1) – reminder

- Pentru fiecare produs din funcția de minimizat se trece 1 in căsuța corespunzătoare din diagramă;
- Se formează suprafețe de minimizare pătrate sau dreptunghiulare alcătuite din 1, 2, 4, 8 sau 16 pătrățele adiacente ce conțin unități;
  - **Observație**: Căsuțele de pe prima linie sunt adiacente cu cele de pe ultima linie, iar cele de pe prima coloană sunt adiacente cu cele de pe ultima coloană.
  - **Proprietăți ale suprafețelor de minimizare**:
    - Trebuie să fie în număr cât mai redus;
    - Trebuie să aibă aria cât mai mare;
    - O căsuță ce conține 1 trebuie să fie conținută în cel puțin o suprafață.

# Diagrame Karnaugh – Algoritm de minimizare (2) – reminder

- Se formează produse pe baza suprafețelor de minimizare astfel:
  - Dacă în interiorul unei suprafețe de minimizare o variabilă își schimbă valoarea, atunci variabila respectivă nu mai apare în produs;
  - Dacă o variabilă își păstrează valoarea 0, atunci ea apare în produs în forma negată;
  - Dacă o variabilă își păstrează valoarea 1, atunci ea apare în produs în forma directă;
- Expresia minimizată este dată de suma produselor obținute la pasul anterior.

# Diagrame Karnaugh – Funcții incomplet specificate – reminder

- Sunt funcții la care nu știm/nu ne interesează valorile de ieșire pentru anumite combinații de intrare
- Există situații când anumite combinații de intrare nu se pot folosi
- Pe viitor, în loc de ? vom folosi fie * fie – → cu aceste poziții facem ce ne convine mai tare astfel încât să obținem minimizarea optimă (se numesc poziții redundante)

# Minimizarea diagramelor Karnaugh cu variabile externe – reminder

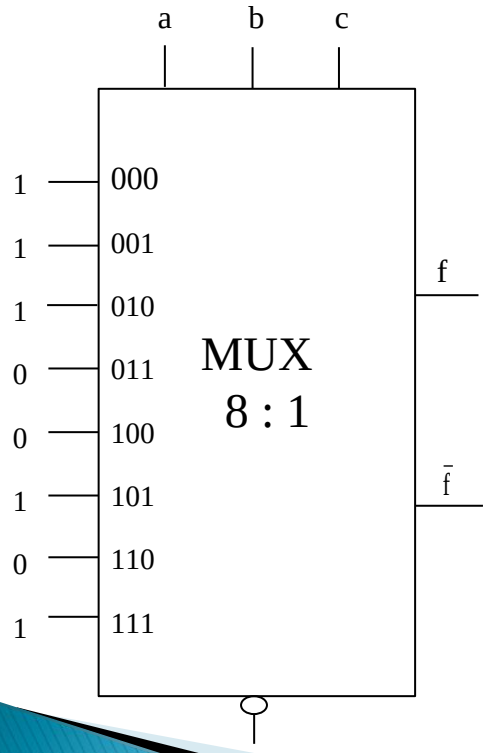
- 1. Toate variabilele externe se consideră = 0 și se minimizează diagrama conform regulilor cunoscute
- 2. Pe rând, fiecare variabilă externă se face = 1, iar unitățile existente în diagramă se consideră redundante (se iau în considerare sau nu în funcție de necesități – similar cu ? de la funcții incomplet specificate). Diagrama se minimizează și apoi rezultatul se înmulțește logic cu variabila externă considerată
- 3. Rezultatul final este dat de suma produselor obținute în pașii anteriori
- Această metodă de minimizare este posibilă prin aplicarea superpoziției (suprapunem rezultatele).

# Minimizarea diagramelor Karnaugh cu variabile externe – Observații – reminder

- În cazul în care avem de minimizat o diagramă ce conține atât o variabilă cât și negatul acesteia, acestea se consideră ca fiind două variabile diferite independente (fără nicio legătură una cu alta).
- În cazul în care avem variabile externe în cadrul funcțiilor incomplet specificate, pozițiile nespecificate pot fi considerate diferit în cei doi pași (pot fi într-un pas 0 și în altul 1, de exemplu).

# MUX – exemplu – reminder

- f(a,b,c) = a!bc + !a!bc + abc + !a!b!c + !ab!c
- Să se implementeze funcția folosind un multiplexor 8:1

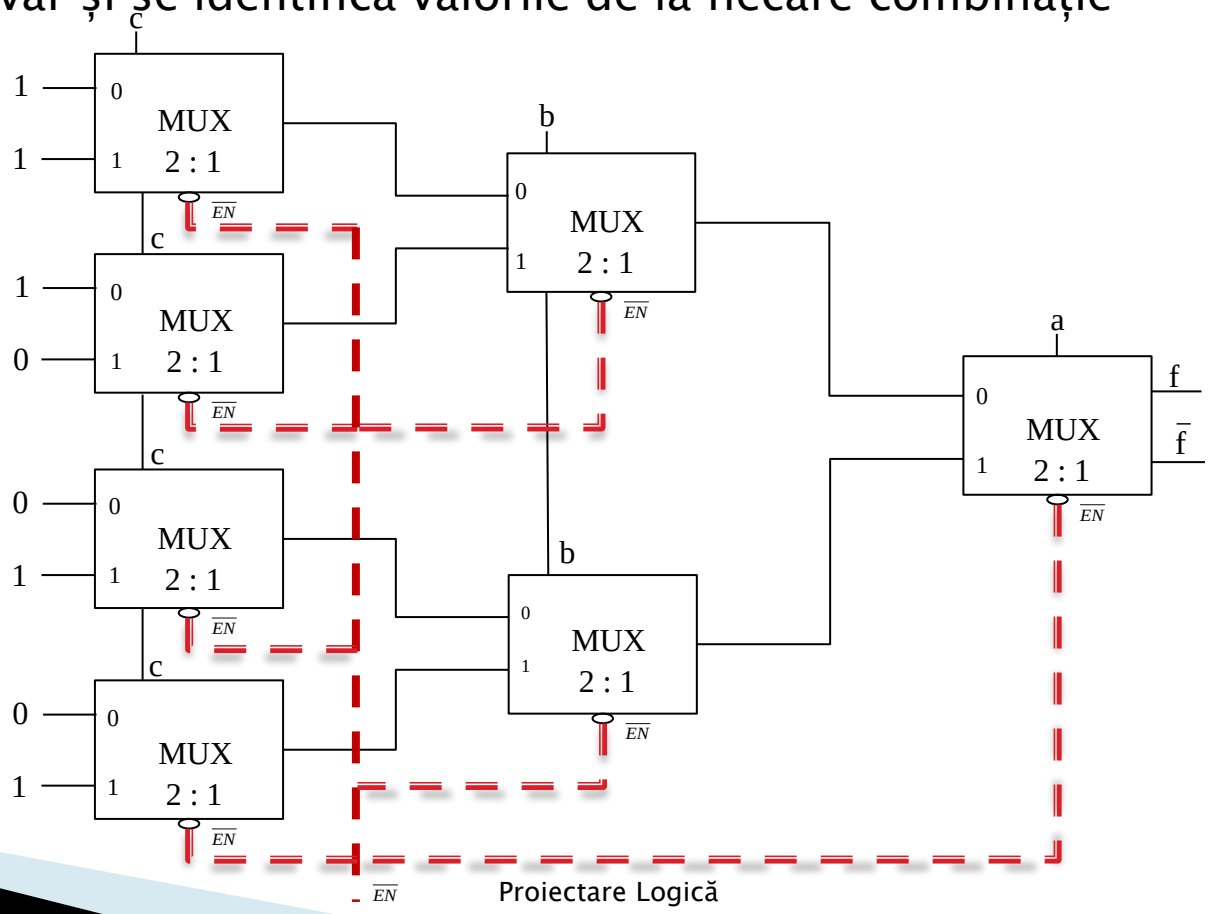



- Cum rezolvăm?
- Scriem forma generală a unei funcții ca sumă de monoame:
- $f(a,b,c) = c_0 * !a!b!c + c_1 * !a!bc + \ldots + c_7 * abc$
- De ce am scris funcția așa?
- Ca să putem identifica coeficienții și să îi punem pe intrările MUX-ului

# MUX – conectare în cascadă – reminder

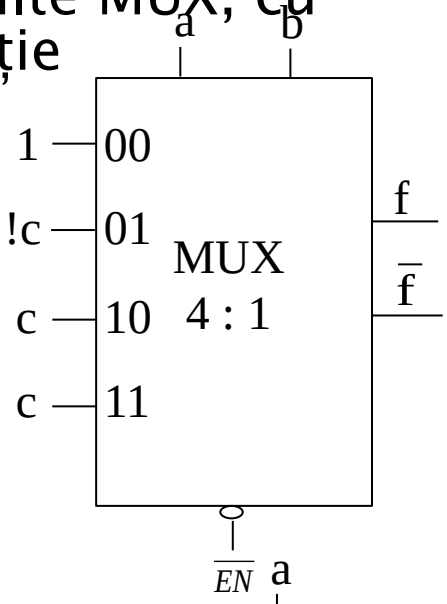
- f(a,b,c) = a!bc + !a!bc + abc + !a!b!c + !ab!c
- Câte MUX 2:1 sunt necesare?
- 4 + 2 + 1
- Cum se completează?
- Se face tabelul de adevăr și se identifică valorile de la fiecare combinație

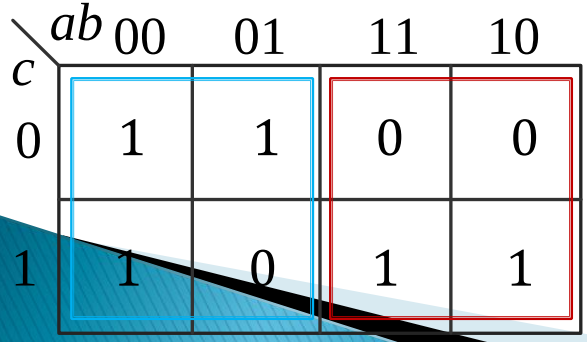
| *a* | *b* | *c* | *f* |
|---|---|---|---|
| 0 | 0 | 0 | 1 |
| 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | 1 |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 0 | 0 |
| 1 | 0 | 1 | 1 |
| 1 | 1 | 0 | 0 |
| 1 | 1 | 1 | 1 |

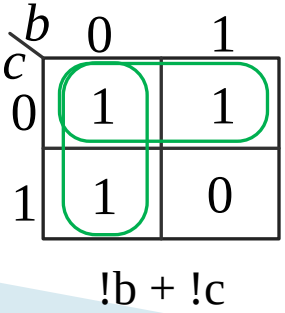


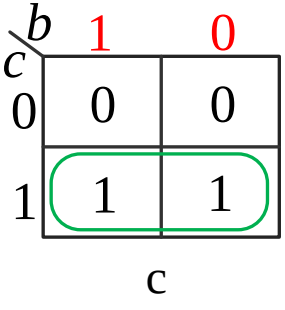
# MUX – multiplexarea intrărilor – reminder

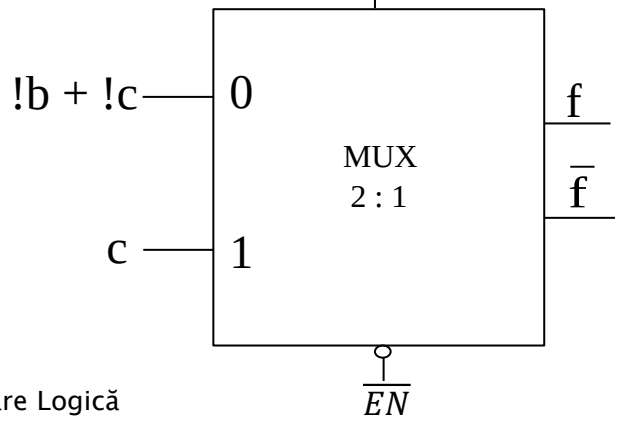
- E greu de realizat asta?
- Nu – iau unul sau mai multe MUX, identific coeficienții și aplic pe intrări valori elementare.
- Problemă: cu cât e mai mare un / sunt mai multe MUX, cu atât e mai scump →implementez aceeași funcție cu un singur MUX 4:1
- Se obține același rezultat dacă pun a, b sau a, c sau b, c pe condiții?
- f(a, b, c) = a!bc + !a!bc + abc + !a!b!c+!ab!c
- f(a, b) = $c_0$*!a!b + $c_1$*!ab+$c_2$*a!b+$c_3$*ab
- Identific coeficienții și completez MUX
- Pot trece și la MUX 2:1 în același fel, dar este mai complicat! → varianta mai simplă e să folosim diagramele Karnaugh



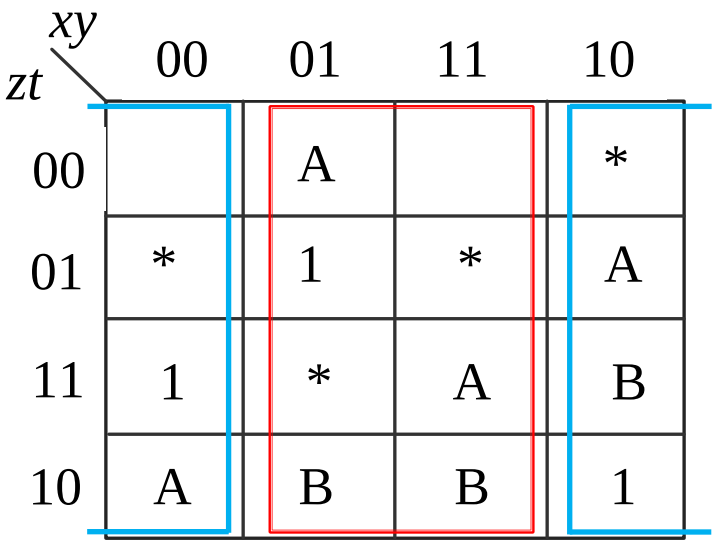
# Terminologie

- Spațiul de adresare este dat de variabilele de selecție ale multiplexorului
  - în exemplul dat a fost {a, b, c}; {a, b}; {a}
- Spațiul de definiție al funcției este dat de variabilele de care depinde funcția
  - în exemplu dat {a, b, c}
- Dacă spațiul de adresare este mai mic decât spațiul de definire al funcției, diferența dintre cele două o constituie variabilele reziduu
  - în exemplu dat {c}; {b, c};

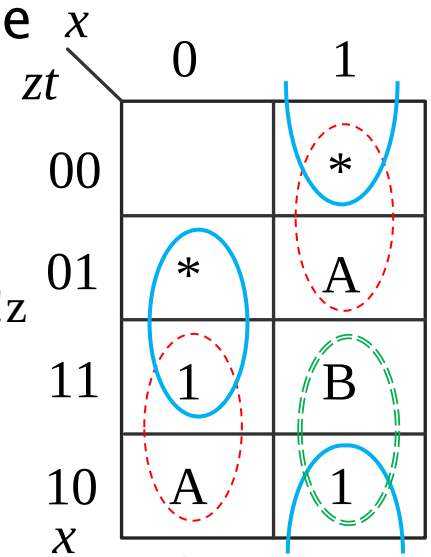
| x | y | z | t | f |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | **0** |
| 0 | 0 | 0 | 1 | * |
| 0 | 0 | 1 | 0 | **A** |
| 0 | 0 | 1 | 1 | **1** |
| 0 | 1 | 0 | 0 | **A** |
| 0 | 1 | 0 | 1 | **1** |
| 0 | 1 | 1 | 0 | **B** |
| 0 | 1 | 1 | 1 | * |
| 1 | 0 | 0 | 0 | * |
| 1 | 0 | 0 | 1 | **A** |
| 1 | 0 | 1 | 0 | **1** |
| 1 | 0 | 1 | 1 | **B** |
| 1 | 1 | 0 | 0 | **0** |
| 1 | 1 | 0 | 1 | * |
| 1 | 1 | 1 | 0 | **B** |
| 1 | 1 | 1 | 1 | **A** |

# Multiplexoare – multiplexarea intrărilor (2)

- Implementați funcția folosind un singur MUX 2:1, având y ca variabilă de selecție

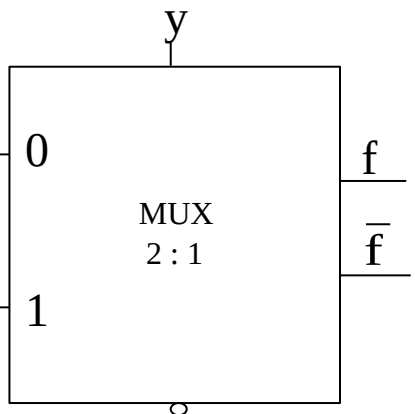



!x*t + x*!t +
A*!x*z + A*x*!z
+ B*x*z

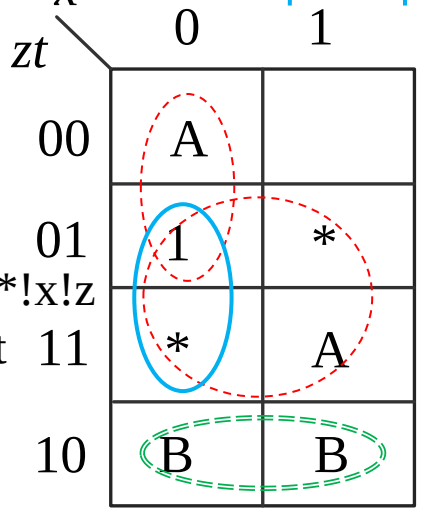



!x*t + x*!t +
A*!x*z + A*x*!z
+ B*x*z

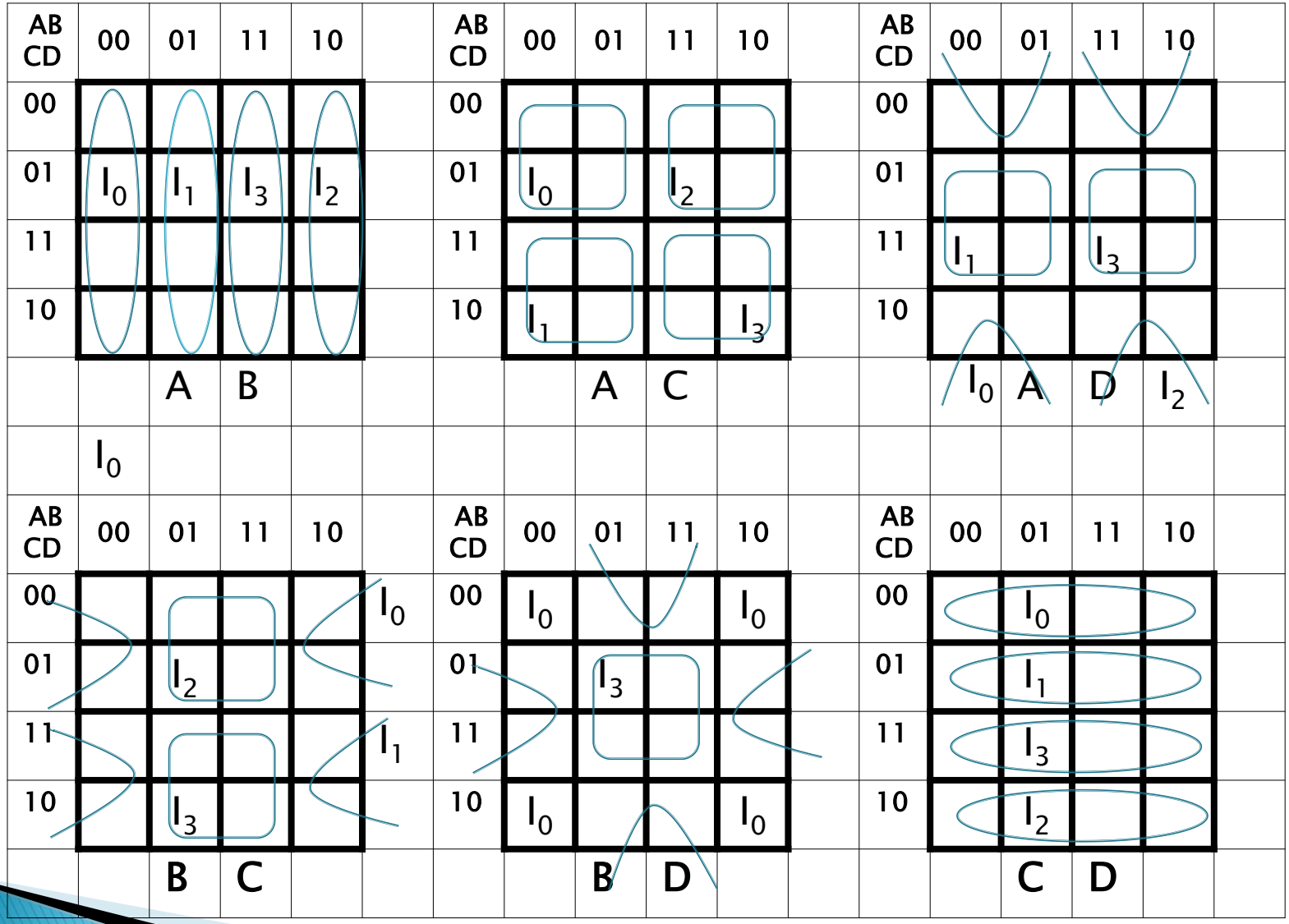
!x*t +
A*t + A*!x!z
+ B*z*!t




!x*t +
A*t + A*!x!z
+ B*z*!t

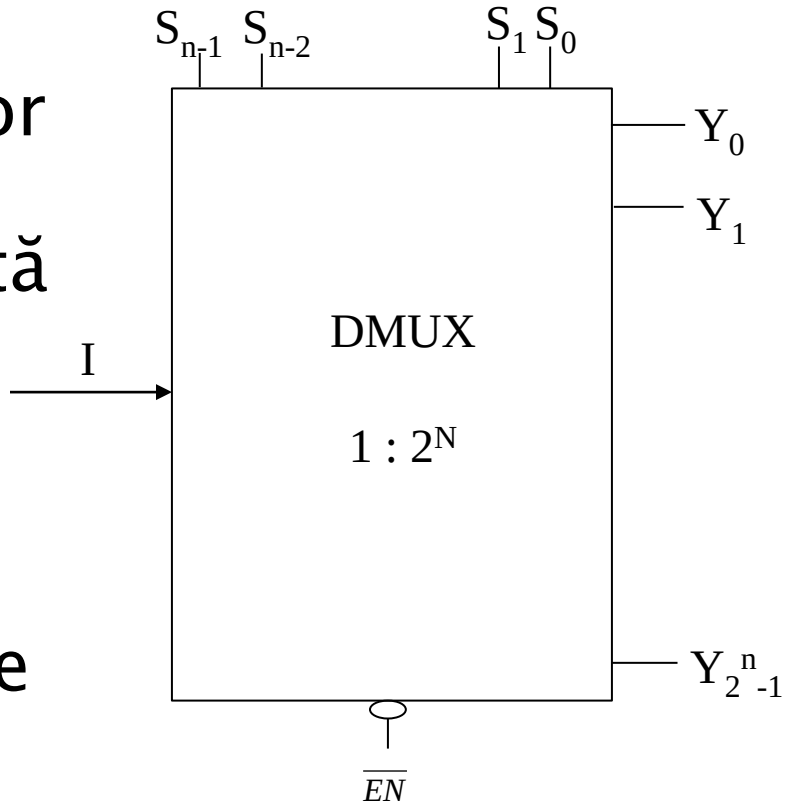


# Spații de adresare și de selecție



# Demultiplexorul

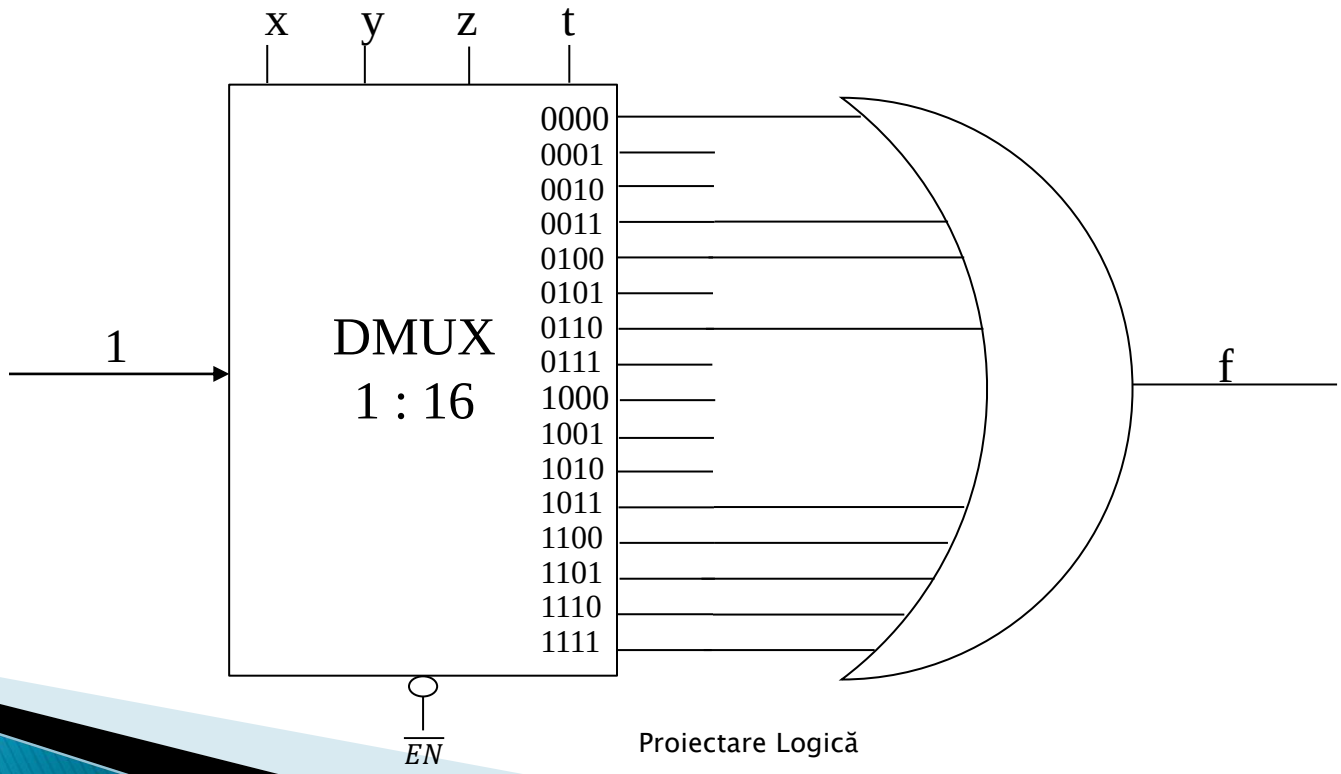
- Circuite complementare multiplexoarelor care permit transmiterea datelor de pe o intrare de date comună pe ieșirea selectată prin cod
- Un demultiplexor are o intrare de date (setată pe „1” logic), o intrare de enable, n intrări de selecție si $2^n$ ieșiri.
- $Y_k = I,\ k = (S_{n-1}\ldots S_0)_{10}$



| x | y | z | t | f |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | **1** |
| 0 | 0 | 0 | 1 | **0** |
| 0 | 0 | 1 | 0 | **0** |
| 0 | 0 | 1 | 1 | **1** |
| 0 | 1 | 0 | 0 | **1** |
| 0 | 1 | 0 | 1 | **0** |
| 0 | 1 | 1 | 0 | **1** |
| 0 | 1 | 1 | 1 | **0** |
| 1 | 0 | 0 | 0 | **0** |
| 1 | 0 | 0 | 1 | **0** |
| 1 | 0 | 1 | 0 | **0** |
| 1 | 0 | 1 | 1 | **1** |
| 1 | 1 | 0 | 0 | **1** |
| 1 | 1 | 0 | 1 | **1** |
| 1 | 1 | 1 | 0 | **1** |
| 1 | 1 | 1 | 1 | **1** |

# Demultiplexoare – exemplu (1)

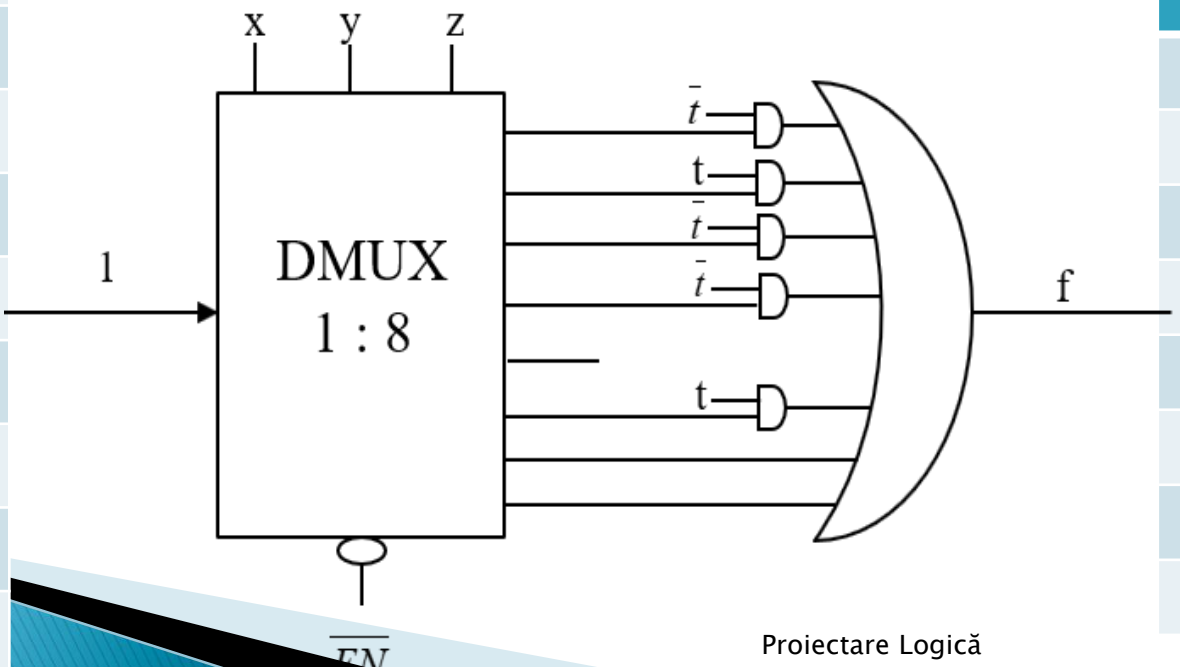
- Implementați funcția folosind DMUX 1:16
- Cum rezolvăm?
- Scriem funcția ca sumă de mintermi
- De ce să scriem funcția așa?
- Pentru sumă există element de circuit – poarta OR!
- Identificăm pentru ce combinații de valori funcția trebuie să ia valoarea „1” și punem ieșirile corespunzătoare într-o poartă de tip OR



# Demultiplexoare – exemplu (2)

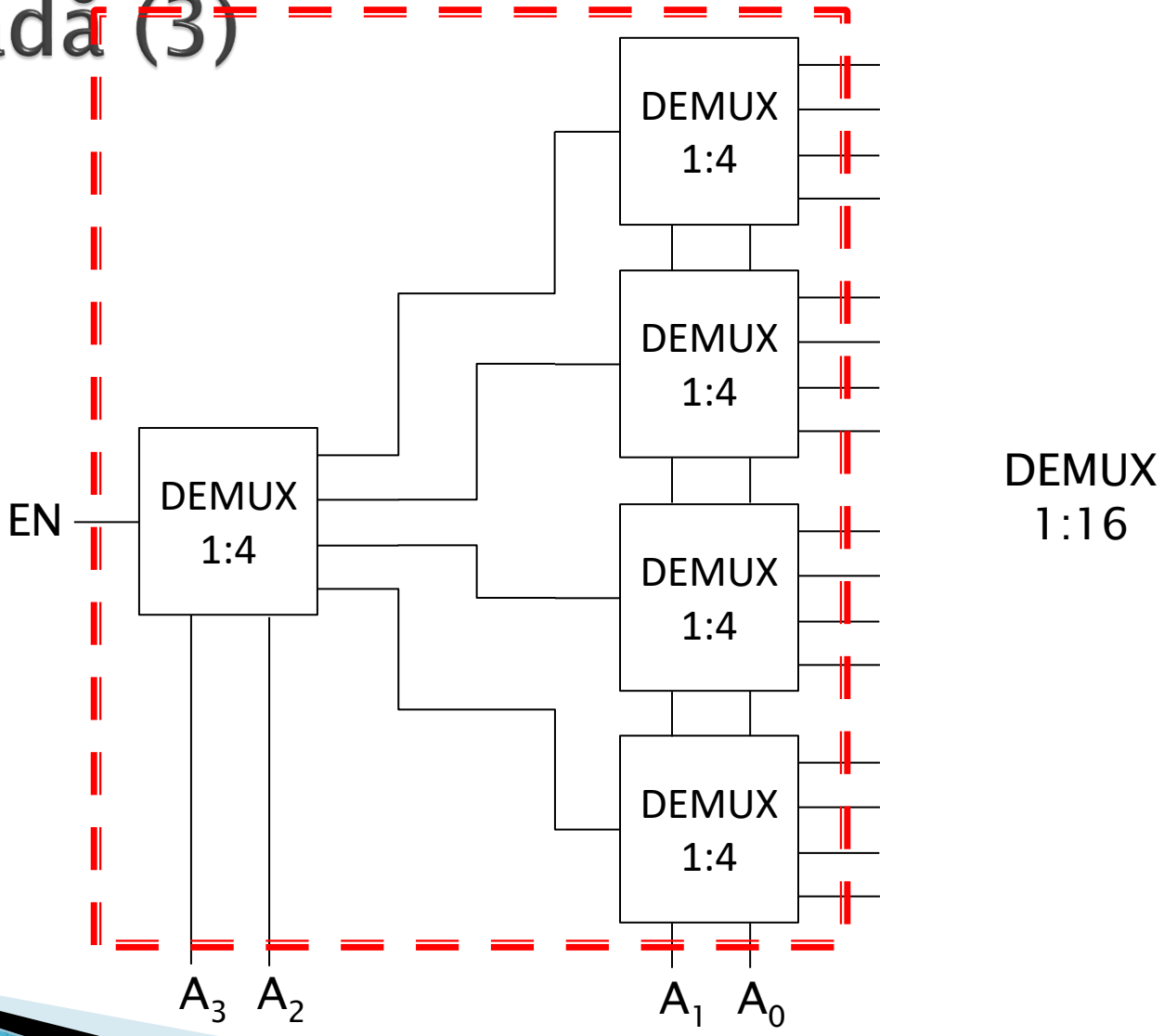
| x | y | z | t | f |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | **1** |
| 0 | 0 | 0 | 1 | **0** |
| 0 | 0 | 1 | 0 | **0** |
| 0 | 0 | 1 | 1 | **1** |
| 0 | 1 | 0 | 0 | **1** |
| 0 | 1 | 0 | 1 | **0** |
| 0 | 1 | 1 | 0 | **1** |
| 0 | 1 | 1 | 1 | **0** |
| 1 | 0 | 0 | 0 | **0** |
| 1 | 0 | 0 | 1 | **0** |
| 1 | 0 | 1 | 0 | **0** |
| 1 | 0 | 1 | 1 | **1** |
| 1 | 1 | 0 | 0 | **1** |
| 1 | 1 | 0 | 1 | **1** |
| 1 | 1 | 1 | 0 | **1** |
| 1 | 1 | 1 | 1 | **1** |

- Implementați funcția folosind DMUX 1:8
- Cum rezolvăm?
- Trecem o variabilă în coloana rezultat și apoi rezolvăm ca în cazul anterior
- Pentru realizarea valorilor ce depind de variabile, se intercalează între demultiplexor și poarta OR porți AND în care se pune variabila respectivă



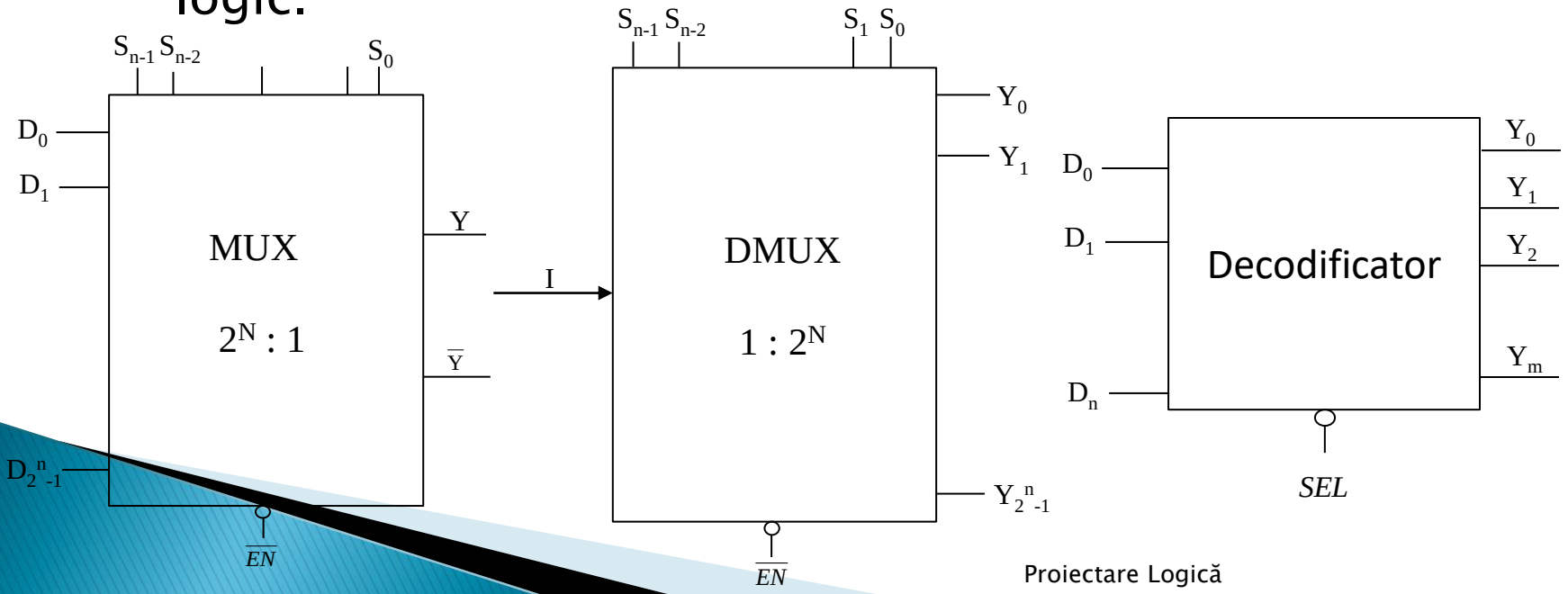

| x | y | z | f |
|---|---|---|---|
| 0 | 0 | 0 | !t |
| 0 | 0 | 1 | t |
| 0 | 1 | 0 | !t |
| 0 | 1 | 1 | !t |
| 1 | 0 | 0 | 0 |
| 1 | 0 | 1 | t |
| 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 |

# Demultiplexoare – conectare în cascadă (3)



# Codificatoare / Decodificatoare (1)

- Circuite combinaționale integrate pe scară medie
- Similare cu MUX/ DMUX
- Majoritatea lucrează în logică negativă (active pe 0)
  - În cazul unui decodificator semnalele de intrare aplicate căilor de adresare determină tranziţia uneia din ieşiri în 0 logic.



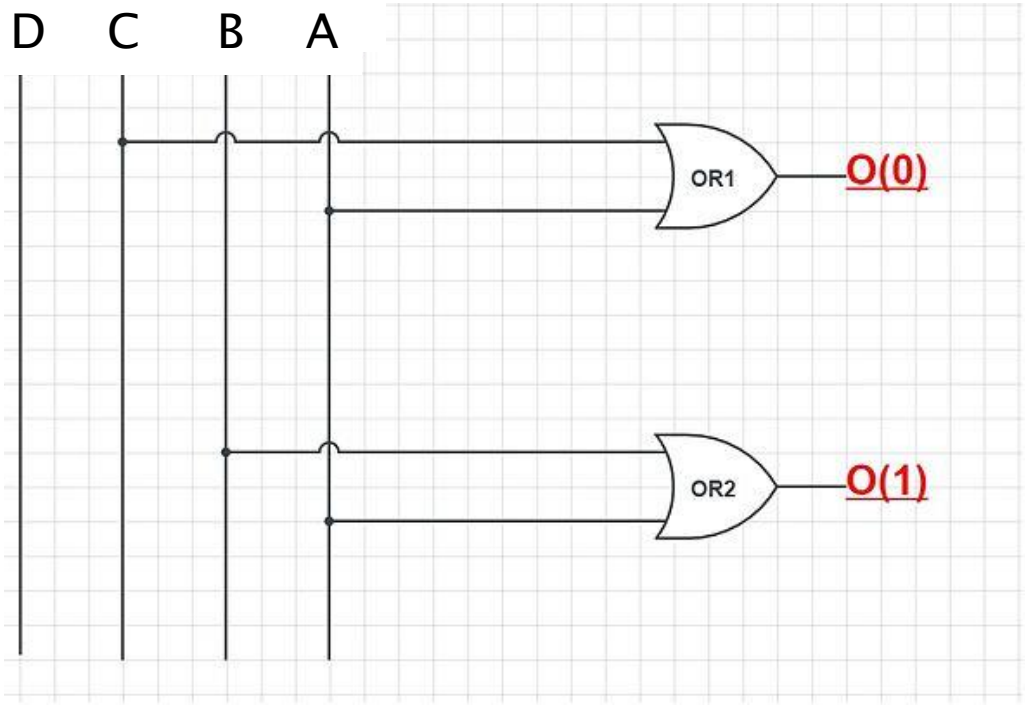
# Codificatoare / Decodificatoare (2)

- *Codificatoare*: Vrem să obținem la ieșire codul binar al cifrei introduse la intrare
- Ex: la tastaturi (telefon)
- *Decodificatoare*: la intrare avem un anumit cod și la ieșire generează semnal pe o singură ieșire, cea corespunzătoare codului de la intrare (de la o reprezentare compactă a informației se obține una și doar una dintre porți activă)
- Ex: instalația de luminițe pentru brad când vrem să aprindem un anumit bec (serie de becuri) și nu alta / toate.

# Codificatoare (1)

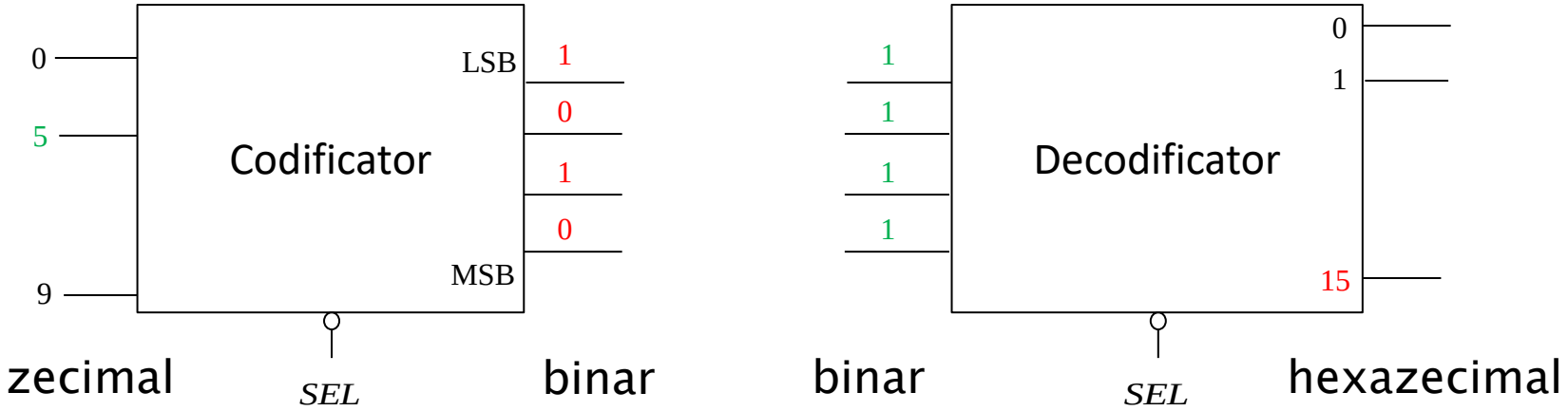
- Un **circuit codificator** atribuie o combinație de valori pentru ieșiri, unică, unui anumit semnal de intrare
- Doar o intrare are valoarea 1, la un moment dat
- Nu este obligatoriu o relație între numărul de intrări și numărul de ieșiri, deși în general avem relația $2^n - n$
- Exemplu codificator 4:2

| A | B | C | D | O(1) | O(0) |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 1 | 0 | 0 |
| 0 | 0 | 1 | 0 | 0 | 1 |
| 0 | 1 | 0 | 0 | 1 | 0 |
| 1 | 0 | 0 | 0 | 1 | 1 |
| Restul combinațiilor nu sunt permise | | | | * | * |



# Codificatoare / Decodificatoare (3)

- E clar principiul de funcționare și diferența dintre ele (și MUX/DMUX)?




- Există o legătură între numărul de intrări și numărul de ieșiri:
  - Mux: $2^{sel}$ = intrări, Decodificatoare: $2^{intrări}$ = ieșiri
  - La codificatoare nu mai există intrări de selecție ci numai un semnal care îl lasă să funcționeze sau nu

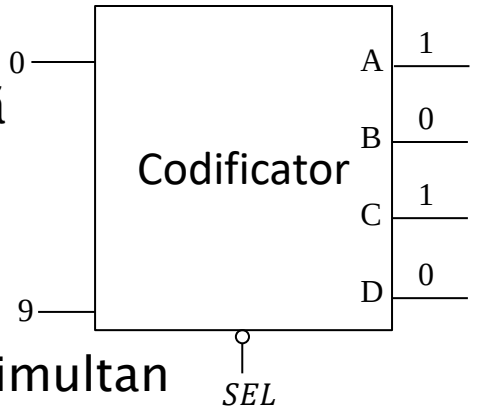
# Codificatoare (2)

- Există codificatoare / decodificatoare binar – hexa, binar – zecimal (ultimele 6 ieșiri nu mai apar), binar –12
- În activitățile curente folosim baza 10 → avem nevoie de codificator zecimal-binar
  - Considerăm logica pozitivă (în cataloage sunt în logică negativă)

| $i_9$ | $i_8$ | $i_7$ | ... | $i_1$ | $i_0$ | D | C | B | A |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | ... | 0 | 0 | * | * | * | * |
| 0 | 0 | 0 | ... | 0 | 1 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | ... | 1 | 0 | 0 | 0 | 0 | 1 |
| 0 | 0 | 0 | ... | 1 | 1 | * | * | * | * |
| | | | | | | | | | |

Câte linii în tabel?
Câte ne interesează
→invalidă

$2^{10}$
10

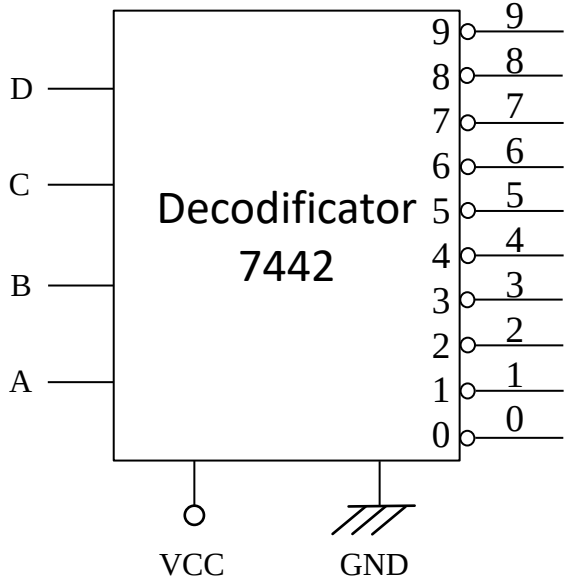



nu apăs 2 taste simultan (primesc semnal pe o singură intrare)

# Decodificatorul 7442

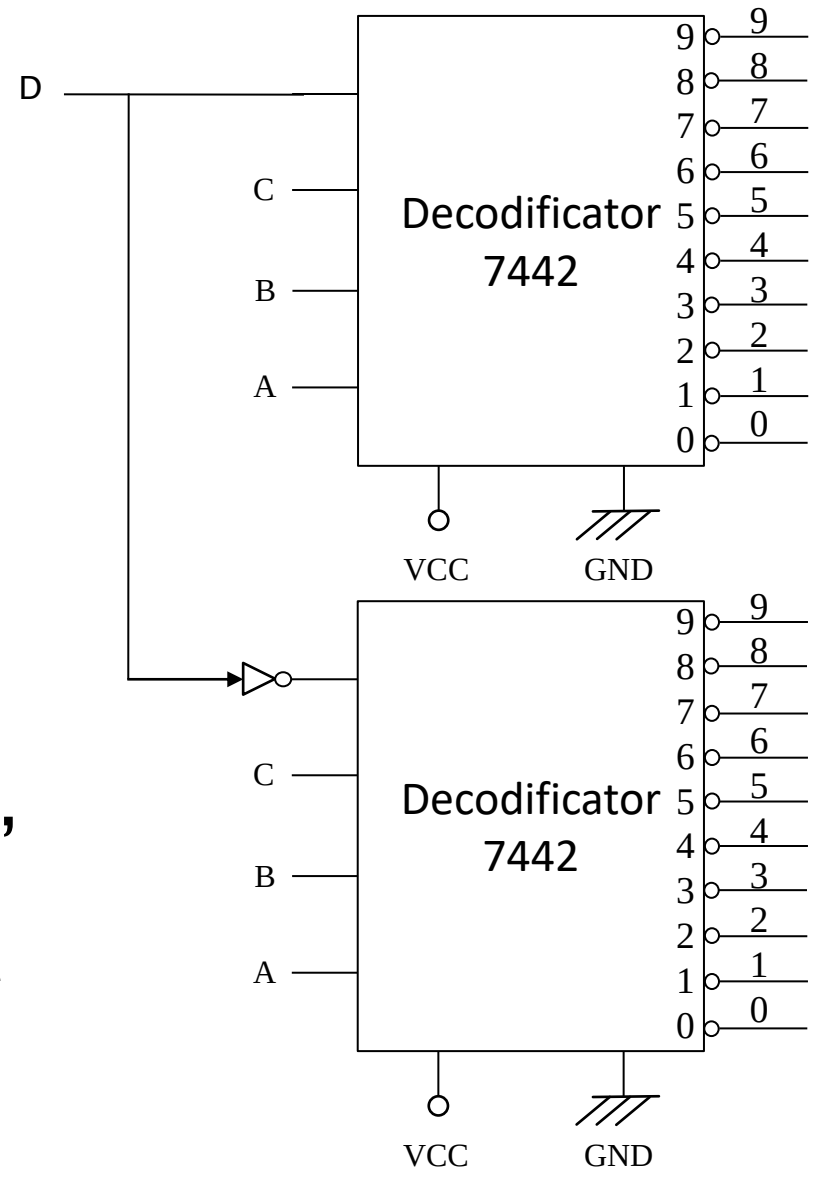
- Decodificator binar – zecimal, activ pe 0
- Nu are nevoie de EN, ci doar de VCC și GND
- Folosit pentru decodificarea spațiului stărilor în circuitele secvențiale și pentru generarea ieșirilor la circuite de tip Moore (le vom studia mai târziu)

| D | C | B | A | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 0 | 0 | 0 | 1 | 1 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 0 | 0 | 1 | 0 | 1 | 1 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 0 | 0 | 1 | 1 | 1 | 1 | 1 | 0 | 1 | 1 | 1 | 1 | 1 | 1 |
| 0 | 1 | 0 | 0 | 1 | 1 | 1 | 1 | 0 | 1 | 1 | 1 | 1 | 1 |
| . | . | . | . | . | . | . | . | . | . | . | . | . | . |



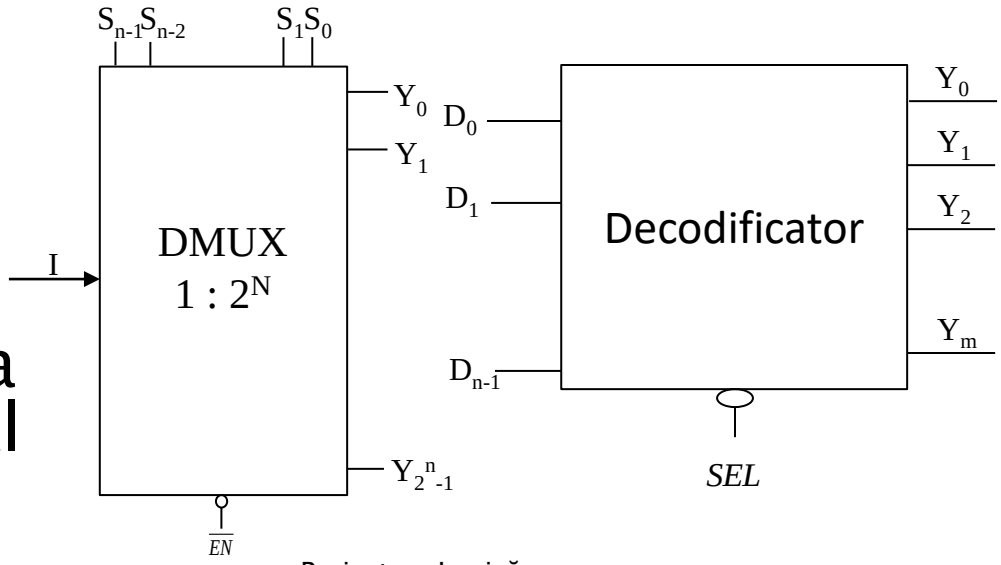
# Decodificatoare

- Insistăm pe decodificator pentru că îl vom utiliza mai târziu la ieșirile circuitelor secvențiale, intrările realizându-se cu multiplexoare.
- Cum putem să scoatem mai multe ieșiri?
- Prin legarea în cascadă (A, B, C) se leagă la ambele aparate, iar D intră o dată direct și o dată negat



# Decodificatoare / demultiplexoare (1)

- Cu excepția lui 7442 care nu are intrări de selecție, decodificatoarele au o intrare de selecție (dacă e activată, atunci în funcție de intrare se activează o ieșire).
- În cataloage nu veți găsi separat decodificatoarele și DMUX, ci le veți găsi în aceeași categorie decodificatoare / DMUX
- Transformarea decodificatorului în DMUX se face prin aplicarea pe intrările de date a variabilelor de selecție și folosirea SEL pe post de semnal de date

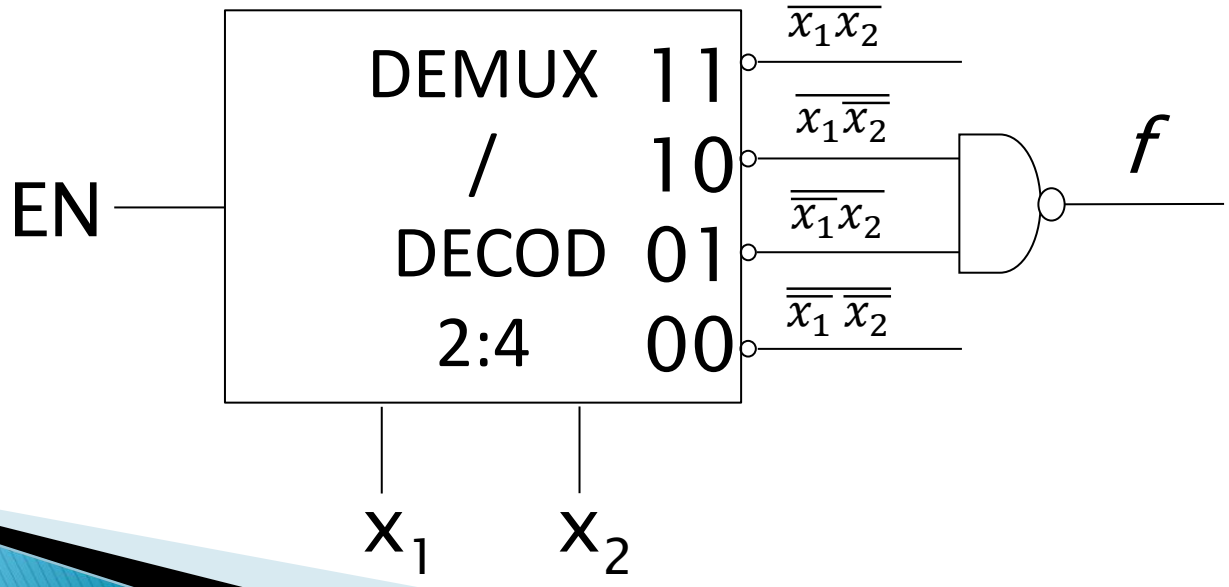


# Decodificatoare / demultiplexoare (2)

- Dacă tot sunt similare, când le folosim pe unele, când pe altele?
- Se preferă utilizarea DMUX în cazul în care nu sunt prea multe funcţii, deoarece chiar dacă folosirea DMUX necesită mai puţine porţi suplimentare, pentru fiecare funcţie este necesar un DMUX.
- Utilizarea unui decodificator se preferă în cazul existenţei multor funcţii deoarece ieşirile unui singur decodificator pot fi folosite pentru generarea unui număr mare de funcţii.

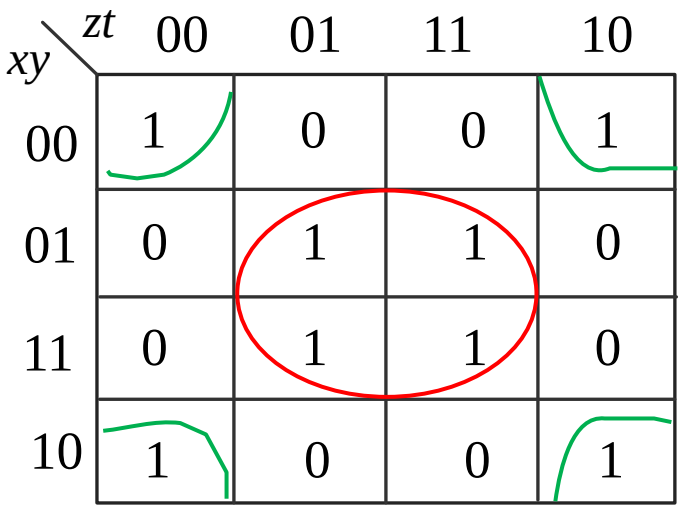
# Folosirea demultiplexoarelor pentru generarea funcțiilor logice

- Se dă funcția: $f(x_1, x_2) = \overline{x_1}x_2 + x_1\overline{x_2} = \overline{\overline{\overline{x_1}x_2}\ \overline{x_1\overline{x_2}}}$
- Să se implementeze funcția folosind DEMUX / DECOD
- Cu DEMUX / DECOD scot semnal pe o singură ieșire la un moment dat, ieșire dată de variabilele de selecție → trebuie să combin ieșirile care îmi trebuie → folosesc NAND (mergea OR dacă erau pozitive)



# Diagrame Karnaugh – Exemplul 1

- Realizați diagrama Karnaugh și apoi minimizați funcția dată prin tabelul de adevăr alăturat.

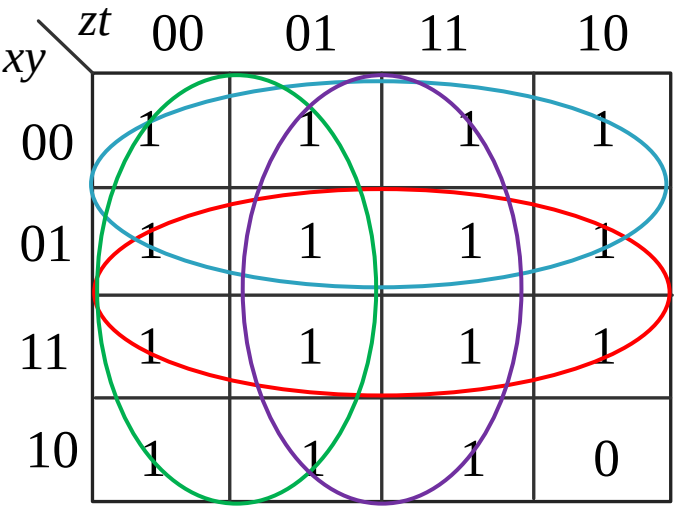


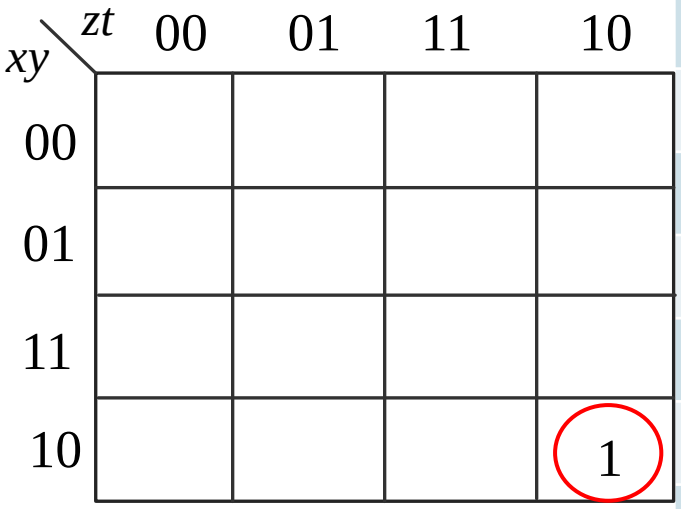

$$f(x,y,z,t) = y * t + \overline{y} * \overline{t}$$

| x | y | z | t | f |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 1 |
| 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 0 | 1 |
| 0 | 0 | 1 | 1 | 0 |
| 0 | 1 | 0 | 0 | 0 |
| 0 | 1 | 0 | 1 | 1 |
| 0 | 1 | 1 | 0 | 0 |
| 0 | 1 | 1 | 1 | 1 |
| 1 | 0 | 0 | 0 | 1 |
| 1 | 0 | 0 | 1 | 0 |
| 1 | 0 | 1 | 0 | 1 |
| 1 | 0 | 1 | 1 | 0 |
| 1 | 1 | 0 | 0 | 0 |
| 1 | 1 | 0 | 1 | 1 |
| 1 | 1 | 1 | 0 | 0 |
| 1 | 1 | 1 | 1 | 1 |

# Diagrame Karnaugh – Exemplul 2

- Realizați diagrama Karnaugh și apoi minimizați funcția dată prin tabelul de adevăr alăturat.




$$f(x,y,z,t) = \overline{x} + y + \overline{z} + t$$

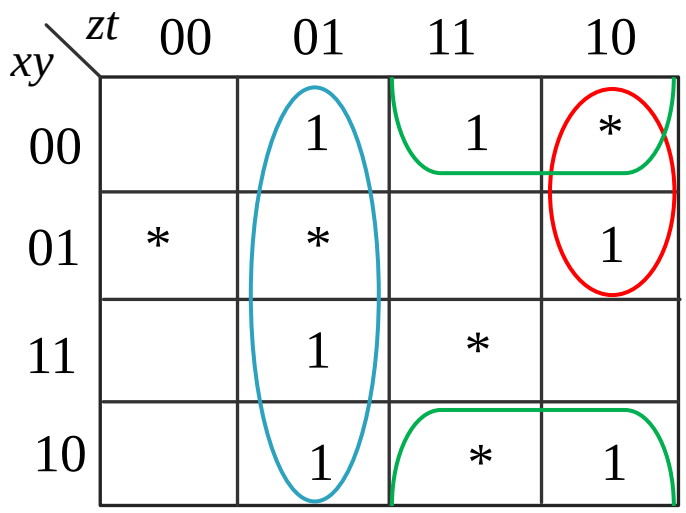
$$\overline{f(x,y,z,t)} = x * \overline{y} * z * \overline{t}$$

$$f(x,y,z,t) = \overline{x * \overline{y} * z * \overline{t}} = \overline{x} + y + \overline{z} + t$$

| x | y | z | t | f | !f |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 0 | 1 | 1 | 0 |
| 0 | 0 | 1 | 0 | 1 | 0 |
| 0 | 0 | 1 | 1 | 1 | 0 |
| 0 | 1 | 0 | 0 | 1 | 0 |
| 0 | 1 | 0 | 1 | 1 | 0 |
| 0 | 1 | 1 | 0 | 1 | 0 |
| 0 | 1 | 1 | 1 | 1 | 0 |
| 1 | 0 | 0 | 0 | 1 | 0 |
| 1 | 0 | 0 | 1 | 1 | 0 |
| 1 | 0 | 1 | 0 | 0 | 1 |
| 1 | 0 | 1 | 1 | 1 | 0 |
| 1 | 1 | 0 | 0 | 1 | 0 |
| 1 | 1 | 0 | 1 | 1 | 0 |
| 1 | 1 | 1 | 0 | 1 | 0 |
| 1 | 1 | 1 | 1 | 1 | 0 |

# Diagrame Karnaugh – Exemplul 3

- Realizați diagrama Karnaugh și apoi minimizați funcția dată prin tabelul de adevăr alăturat.



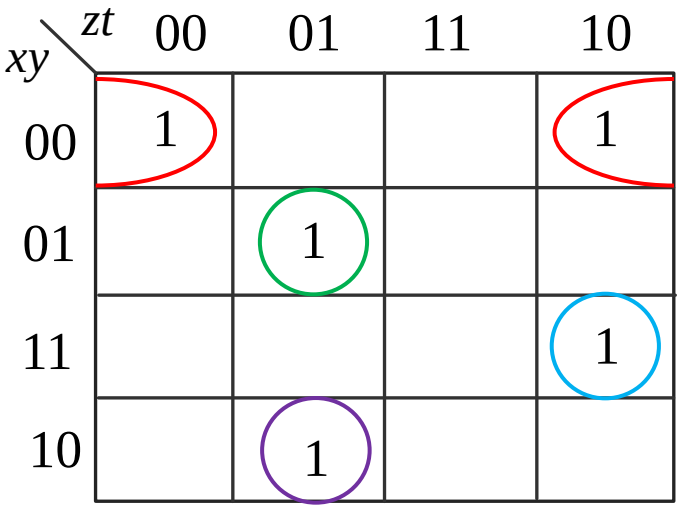

$$f(x,y,z,t) = \overline{z} * t + \overline{y} * z + \overline{x} * z * \overline{t}$$

| x | y | z | t | f |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | 1 |
| 0 | 0 | 1 | 0 | * |
| 0 | 0 | 1 | 1 | 1 |
| 0 | 1 | 0 | 0 | * |
| 0 | 1 | 0 | 1 | * |
| 0 | 1 | 1 | 0 | 1 |
| 0 | 1 | 1 | 1 | 0 |
| 1 | 0 | 0 | 0 | 0 |
| 1 | 0 | 0 | 1 | 1 |
| 1 | 0 | 1 | 0 | 1 |
| 1 | 0 | 1 | 1 | * |
| 1 | 1 | 0 | 0 | 0 |
| 1 | 1 | 0 | 1 | 1 |
| 1 | 1 | 1 | 0 | 0 |
| 1 | 1 | 1 | 1 | * |

# Diagrame Karnaugh – Exemplul 4 (1)

- Realizați diagrama Karnaugh și apoi minimizați funcția dată prin tabelul de adevăr alăturat.

| xy \ zt | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | 1 | A | !A | 1 |
| 01 | !A | 1 | A | |
| 11 | | !A | !A | 1 |
| 10 | A | 1 | !A | |

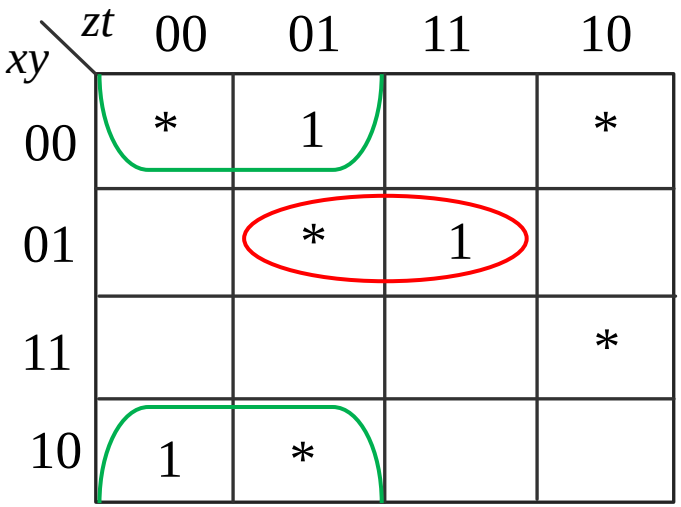


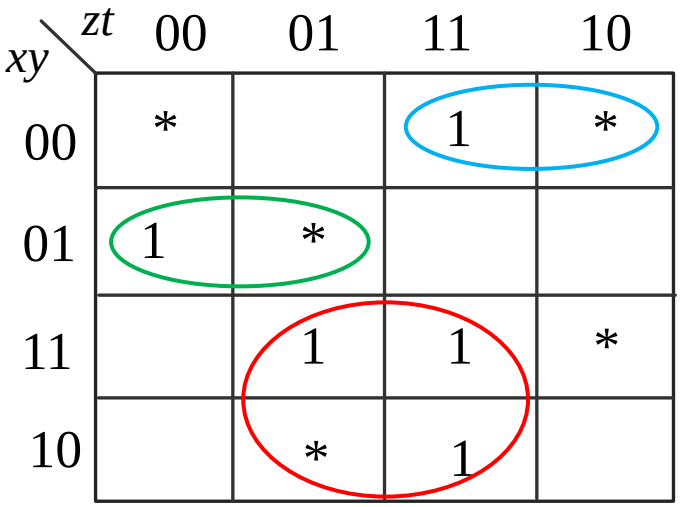

$$f'(x,y,z,t,A,\overline{A}) = \overline{x} * \overline{y} * \overline{t} + \overline{x} * y * \overline{z} * t$$
$$+x * y * z * \overline{t} + x * \overline{y} * \overline{z} * t$$

| x | y | z | t | f |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 1 |
| 0 | 0 | 0 | 1 | A |
| 0 | 0 | 1 | 0 | 1 |
| 0 | 0 | 1 | 1 | !A |
| 0 | 1 | 0 | 0 | !A |
| 0 | 1 | 0 | 1 | 1 |
| 0 | 1 | 1 | 0 | 0 |
| 0 | 1 | 1 | 1 | A |
| 1 | 0 | 0 | 0 | A |
| 1 | 0 | 0 | 1 | 1 |
| 1 | 0 | 1 | 0 | 0 |
| 1 | 0 | 1 | 1 | !A |
| 1 | 1 | 0 | 0 | 0 |
| 1 | 1 | 0 | 1 | !A |
| 1 | 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 | !A |

# Diagrame Karnaugh – Exemplul 4 (2)

- Realizați diagrama Karnaugh și apoi minimizați funcția dată prin tabelul de adevăr alăturat.




$$f''(x,y,z,t,A,\overline{A}) = \overline{y} * \overline{z} * A + \overline{x} * y * t * A$$

$$f'''(x,y,z,t,A,\overline{A}) = \overline{x} * \overline{y} * z * \overline{A} + \overline{x} * y * \overline{z} * \overline{A} + x * t * \overline{A}$$

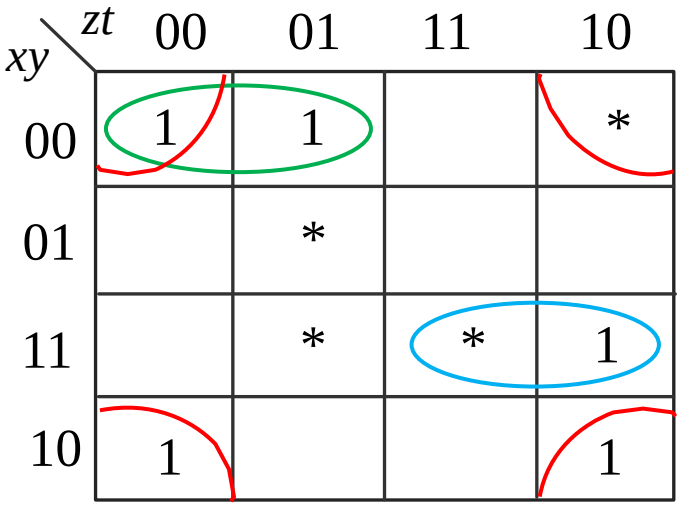
$$f(x,y,z,t,A,\overline{A}) = f' + f'' + f'''$$

| x | y | z | t | f |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 1 |
| 0 | 0 | 0 | 1 | A |
| 0 | 0 | 1 | 0 | 1 |
| 0 | 0 | 1 | 1 | !A |
| 0 | 1 | 0 | 0 | !A |
| 0 | 1 | 0 | 1 | 1 |
| 0 | 1 | 1 | 0 | 0 |
| 0 | 1 | 1 | 1 | A |
| 1 | 0 | 0 | 0 | A |
| 1 | 0 | 0 | 1 | 1 |
| 1 | 0 | 1 | 0 | 0 |
| 1 | 0 | 1 | 1 | !A |
| 1 | 1 | 0 | 0 | 0 |
| 1 | 1 | 0 | 1 | !A |
| 1 | 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 | !A |

# Diagrame Karnaugh – Exemplul 5 (1)

- Realizați diagrama Karnaugh și apoi minimizați funcția dată prin tabelul de adevăr alăturat. În final, implementați circuitul folosind circuite de tip NAND:

| xy \ zt | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | 1 | 1 | A | * |
| 01 | | * | | B |
| 11 | A | * | * | 1 |
| 10 | 1 | A | B | 1 |



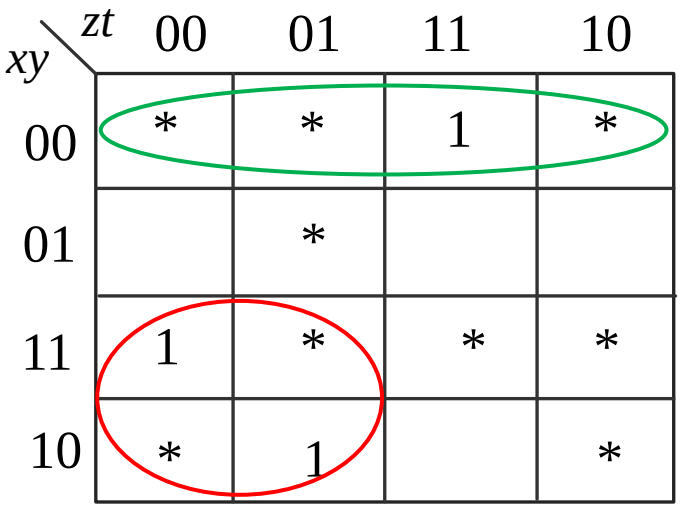

$$f_1 = \overline{y} * \overline{t} + \overline{x} * \overline{y} * \overline{z} + x * y * z$$

| x | y | z | t | f |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 1 |
| 0 | 0 | 0 | 1 | 1 |
| 0 | 0 | 1 | 0 | * |
| 0 | 0 | 1 | 1 | A |
| 0 | 1 | 0 | 0 | 0 |
| 0 | 1 | 0 | 1 | * |
| 0 | 1 | 1 | 0 | B |
| 0 | 1 | 1 | 1 | 0 |
| 1 | 0 | 0 | 0 | 1 |
| 1 | 0 | 0 | 1 | A |
| 1 | 0 | 1 | 0 | 1 |
| 1 | 0 | 1 | 1 | B |
| 1 | 1 | 0 | 0 | A |
| 1 | 1 | 0 | 1 | * |
| 1 | 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 | * |

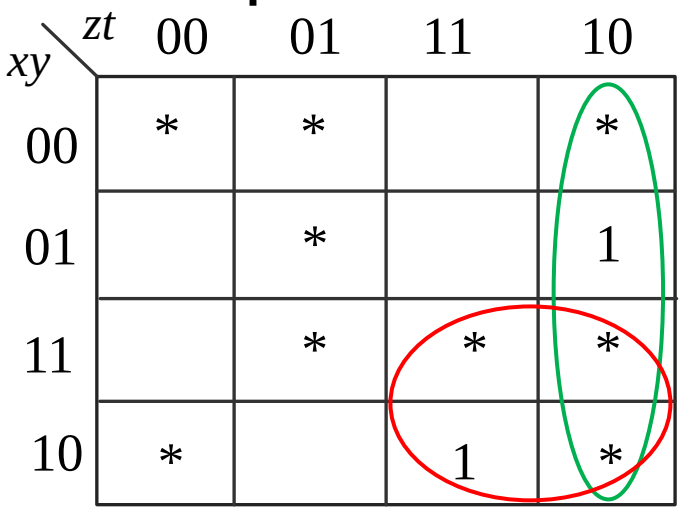
# Diagrame Karnaugh – Exemplul 5 (2)

- Realizați diagrama Karnaugh și apoi minimizați funcția dată prin tabelul de adevăr alăturat. În final, implementați circuitul folosind circuite de tip NAND:

| x | y | z | t | f |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 1 |
| 0 | 0 | 0 | 1 | 1 |
| 0 | 0 | 1 | 0 | * |
| 0 | 0 | 1 | 1 | A |
| 0 | 1 | 0 | 0 | 0 |
| 0 | 1 | 0 | 1 | * |
| 0 | 1 | 1 | 0 | B |
| 0 | 1 | 1 | 1 | 0 |
| 1 | 0 | 0 | 0 | 1 |
| 1 | 0 | 0 | 1 | A |
| 1 | 0 | 1 | 0 | 1 |
| 1 | 0 | 1 | 1 | B |
| 1 | 1 | 0 | 0 | A |
| 1 | 1 | 0 | 1 | * |
| 1 | 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 | * |




$$f_2 = \overline{x} * \overline{y} * A + x * \overline{z} * A$$




$$f_3 = z * \overline{t} * B + x * z * B$$

# Diagrame Karnaugh – Exemplul 5 (3)

$$f_1 = \overline{y} * \overline{t} + \overline{x} * \overline{y} * \overline{z} + x * y * z$$

$$f_2 = \overline{x} * \overline{y} * A + x * \overline{z} * A$$

$$f_3 = z * \overline{t} * B + x * z * B$$

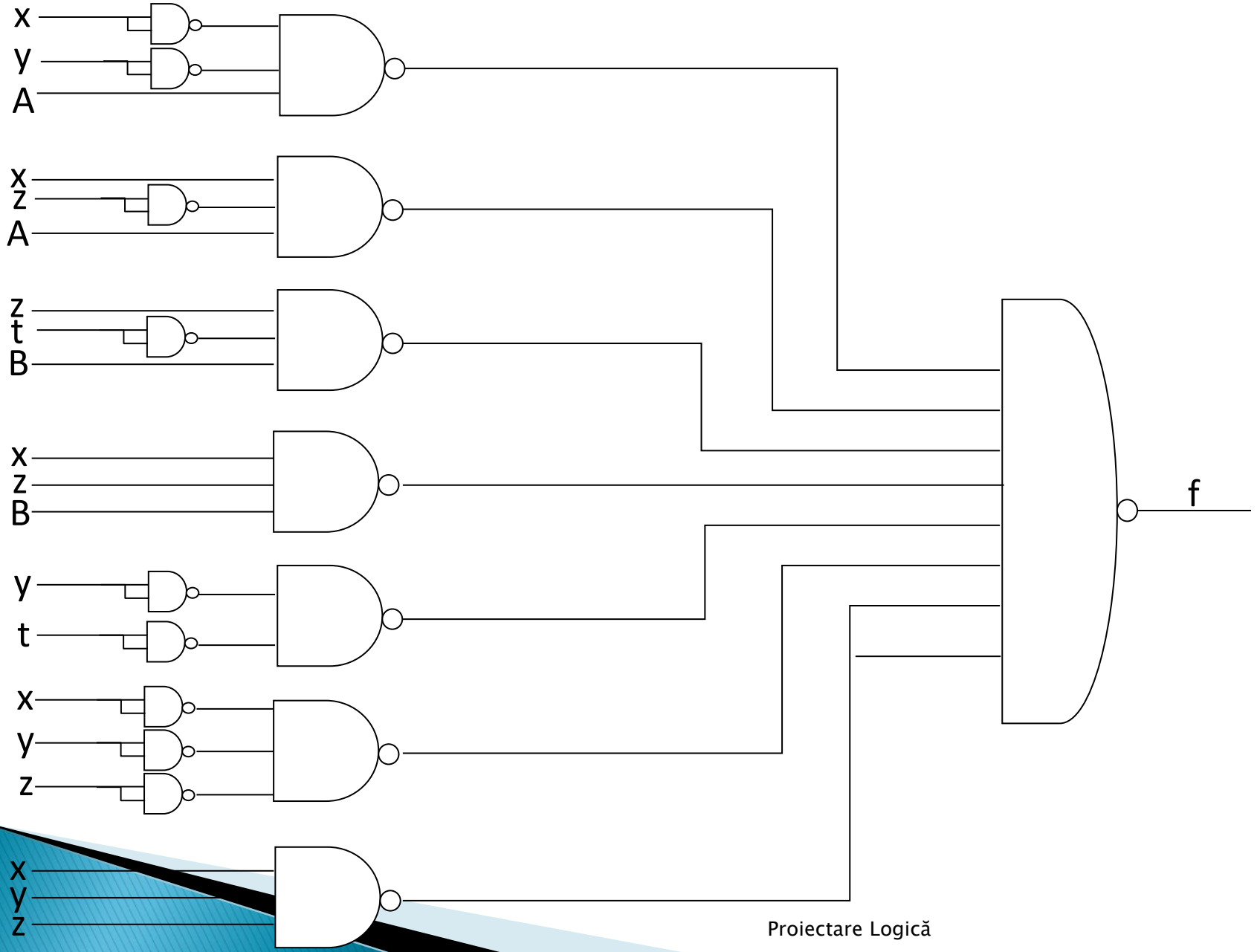
$$f(x, y, z, t, A, B) = f_1 + f_2 + f_3$$

$$f(x, y, z, t, A, B) = \overline{y} * \overline{t} + \overline{x} * \overline{y} * \overline{z} + x * y * z + \overline{x} * \overline{y} * A + x * \overline{z} * A + z * \overline{t} * B + x * z * B$$

$$f(x, y, z, t, A, B) = \overline{\overline{y} * \overline{t} + \overline{x} * \overline{y} * \overline{z} + x * y * z + \overline{x} * \overline{y} * A + x * \overline{z} * A + z * \overline{t} * B + x * z * B}$$

$$f(x, y, z, t, A, B) = \overline{\overline{y} * \overline{t} * \overline{\overline{x} * \overline{y} * \overline{z}} * \overline{x * y * z} * \overline{\overline{x} * \overline{y} * A} * \overline{x * \overline{z} * A} * \overline{z * \overline{t} * B} * \overline{x * z * B}}$$

# Diagrame Karnaugh – Exemplul 5 (4)



# Diagrame Karnaugh – Exemplul 6 (1)

- Realizați diagrama Karnaugh și apoi minimizați funcția dată prin tabelul de adevăr alăturat. În final, implementați circuitul folosind circuite de tip NAND:

| xy \ zt | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | | * | 1 | A |
| 01 | A | 1 | * | B |
| 11 | | * | A | B |
| 10 | * | A | B | 1 |

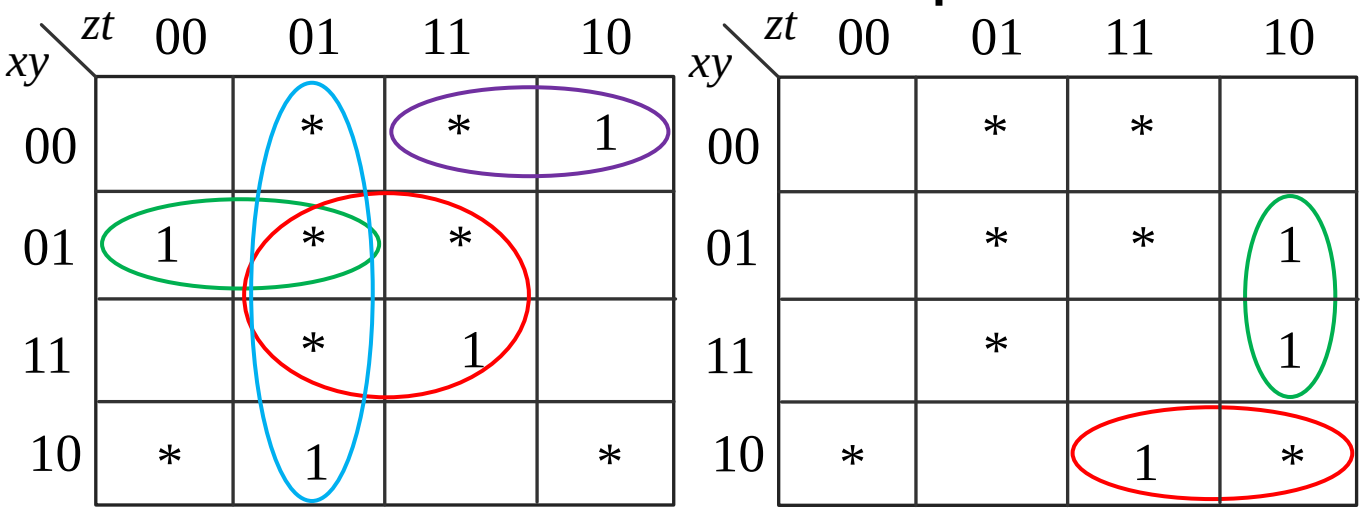
| xy \ zt | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | | * | 1 | |
| 01 | | 1 | * | |
| 11 | | * | | |
| 10 | * | | | 1 |

$$f_1 = \overline{x} * t + x * \overline{y} * \overline{t}$$

| x | y | z | t | f |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | * |
| 0 | 0 | 1 | 0 | A |
| 0 | 0 | 1 | 1 | 1 |
| 0 | 1 | 0 | 0 | A |
| 0 | 1 | 0 | 1 | 1 |
| 0 | 1 | 1 | 0 | B |
| 0 | 1 | 1 | 1 | * |
| 1 | 0 | 0 | 0 | * |
| 1 | 0 | 0 | 1 | A |
| 1 | 0 | 1 | 0 | 1 |
| 1 | 0 | 1 | 1 | B |
| 1 | 1 | 0 | 0 | 0 |
| 1 | 1 | 0 | 1 | * |
| 1 | 1 | 1 | 0 | B |
| 1 | 1 | 1 | 1 | A |

# Diagrame Karnaugh – Exemplul 6 (2)

- Realizați diagrama Karnaugh și apoi minimizați funcția dată prin tabelul de adevăr alăturat. În final, implementați circuitul folosind circuite de tip NAND:




$$f_2 = \overline{z} * t * A + y * t * A + \overline{x} * y * \overline{z} * A + \overline{x} * \overline{y} * z * A$$

$$f_3 = y * z * \overline{t} * B + x * \overline{y} * z * B$$

| x | y | z | t | f |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | * |
| 0 | 0 | 1 | 0 | A |
| 0 | 0 | 1 | 1 | 1 |
| 0 | 1 | 0 | 0 | A |
| 0 | 1 | 0 | 1 | 1 |
| 0 | 1 | 1 | 0 | B |
| 0 | 1 | 1 | 1 | * |
| 1 | 0 | 0 | 0 | * |
| 1 | 0 | 0 | 1 | A |
| 1 | 0 | 1 | 0 | 1 |
| 1 | 0 | 1 | 1 | B |
| 1 | 1 | 0 | 0 | 0 |
| 1 | 1 | 0 | 1 | * |
| 1 | 1 | 1 | 0 | B |
| 1 | 1 | 1 | 1 | A |

# Diagrame Karnaugh – Exemplul 5 (3)

$$f_1 = \overline{x} * t + x * \overline{y} * \overline{t}$$

$$f_2 = \overline{z} * t * A + y * t * A + \overline{x} * y * \overline{z} * A + \overline{x} * \overline{y} * z * A$$

$$f_3 = y * z * \overline{t} * B + x * \overline{y} * z * B$$

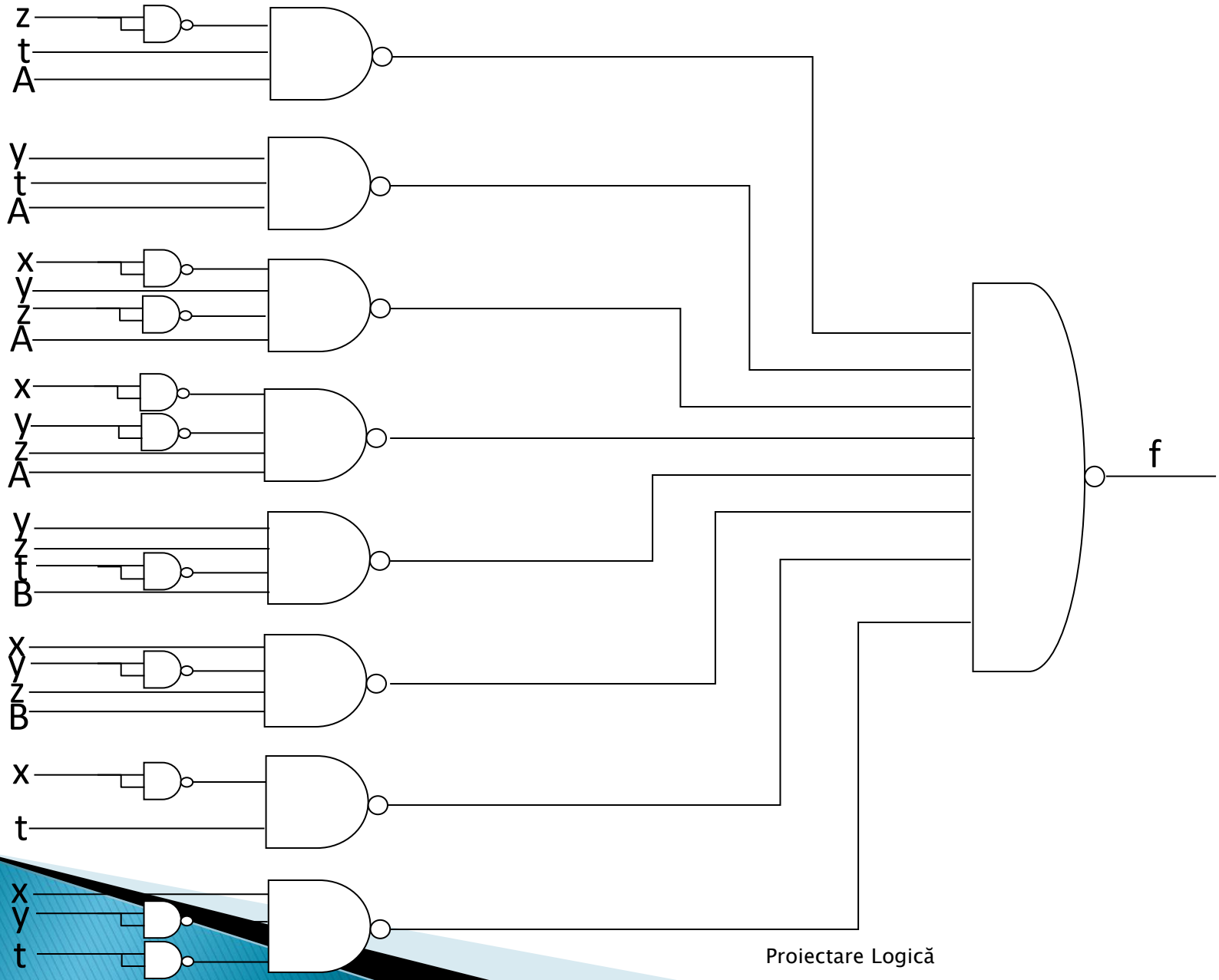
$$f(x, y, z, t, A, B) = f_1 + f_2 + f_3$$

$$f(x, y, z, t, A, B) = \overline{x} * t + x * \overline{y} * \overline{t} + \overline{z} * t * A + y * t * A + \overline{x} * y * \overline{z} * A + \overline{x} * \overline{y} * z * A + y * z * \overline{t} * B + x * \overline{y} * z * B$$

$$f(x, y, z, t, A, B) = \overline{\overline{x} * t + x * \overline{y} * \overline{t} + \overline{z} * t * A + y * t * A + \overline{x} * y * \overline{z} * A + \overline{x} * \overline{y} * z * A + y * z * \overline{t} * B + x * \overline{y} * z * B}$$

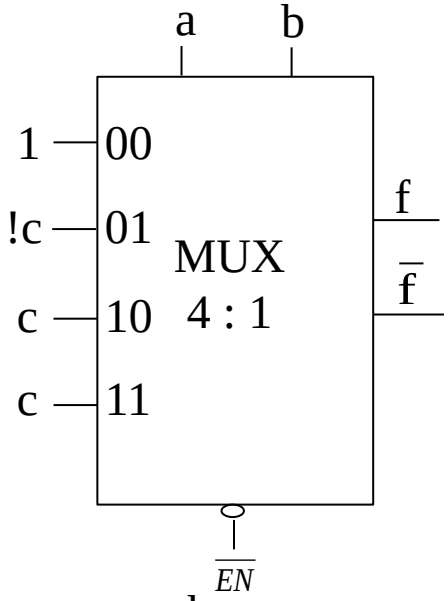
$$f(x, y, z, t, A, B) = \overline{\overline{x} * t} * \overline{x * \overline{y} * \overline{t}} * \overline{\overline{z} * t * A} * \overline{y * t * A} * \overline{\overline{x} * y * \overline{z} * A} * \overline{\overline{x} * \overline{y} * z * A} * \overline{y * z * \overline{t} * B} * \overline{x * \overline{y} * z * B}$$

# Diagrame Karnaugh – Exemplul 6 (4)

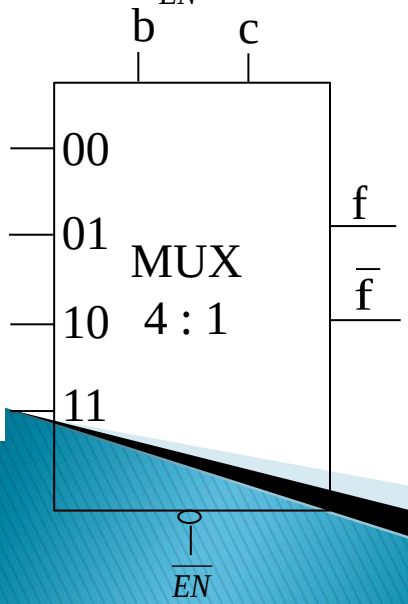


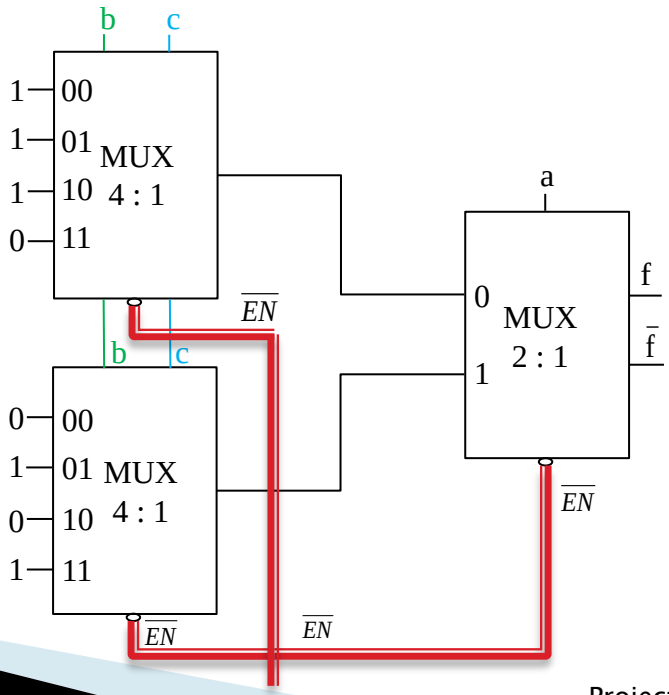
# Aplicație multiplexoare

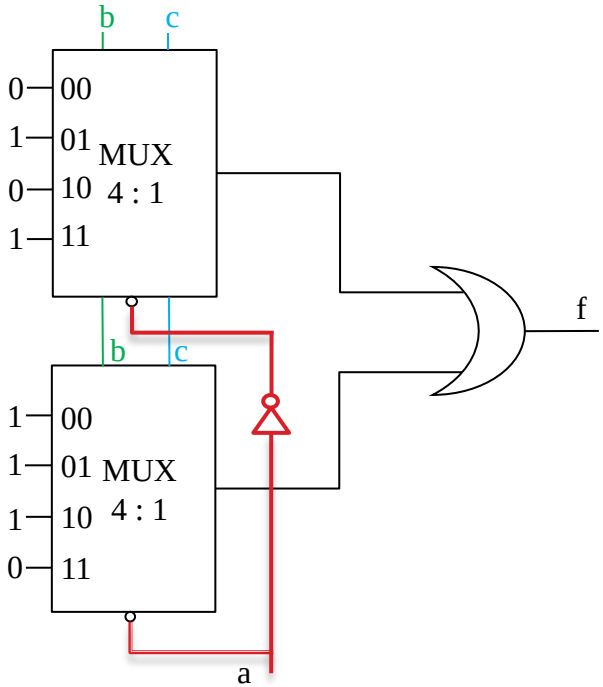
$f(a,b,c) = a!bc + !a!bc + abc + !a!b!c + !ab!c$




| $ab$ / $c$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 0 | 1 | 1 | 0 | 0 |
| 1 | 1 | 0 | 1 | 1 |

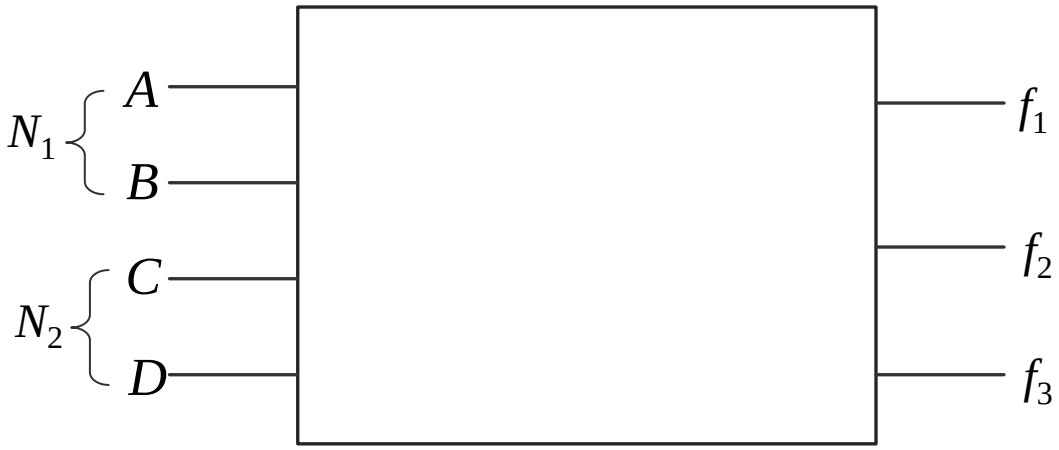


# Diagrame Karnaugh – Aplicația 1: comparator pe 2 biți

Comparator pe 2 biți




Egal cu 1 dacă

$(N_1 < N_2)$

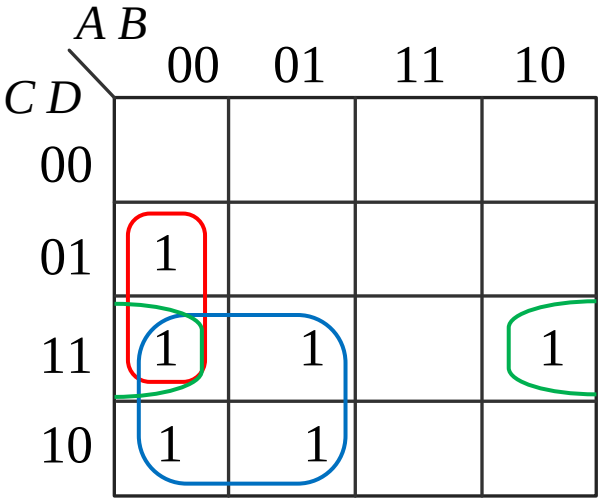
$(N_1 = N_2)$

$(N_1 > N_2)$

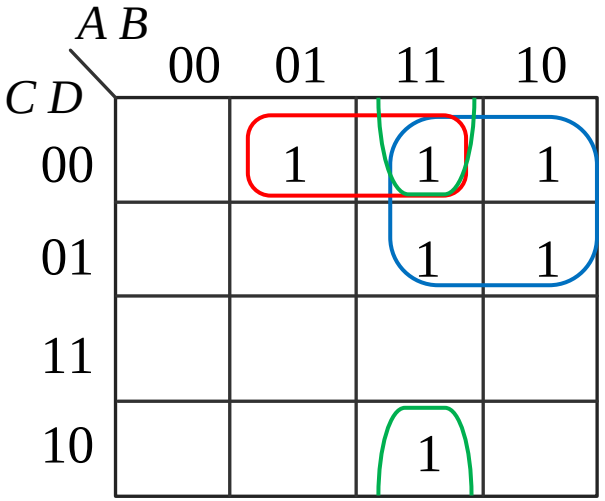
- Pornim de la tabela de adevăr

# Comparator pe 2 biți (1)

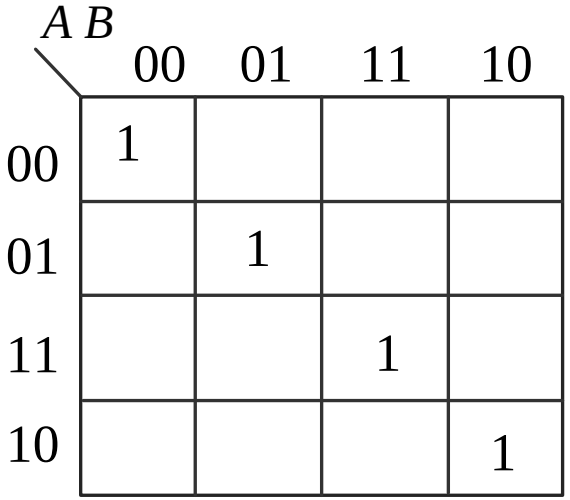
| $A$ | $B$ | $C$ | $D$ | $f_1$ | $f_2$ | $f_3$ |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | | | |
| 0 | 0 | 0 | 1 | | | |
| 0 | 0 | 1 | 0 | | | |
| 0 | 0 | 1 | 1 | | | |
| 0 | 1 | 0 | 0 | | | |
| 0 | 1 | 0 | 1 | | | |
| 0 | 1 | 1 | 0 | | | |
| 0 | 1 | 1 | 1 | | | |
| 1 | 0 | 0 | 0 | | | |
| 1 | 0 | 0 | 1 | | | |
| 1 | 0 | 1 | 0 | | | |
| 1 | 0 | 1 | 1 | | | |
| 1 | 1 | 0 | 0 | | | |
| 1 | 1 | 0 | 1 | | | |
| 1 | 1 | 1 | 0 | | | |
| 1 | 1 | 1 | 1 | | | |



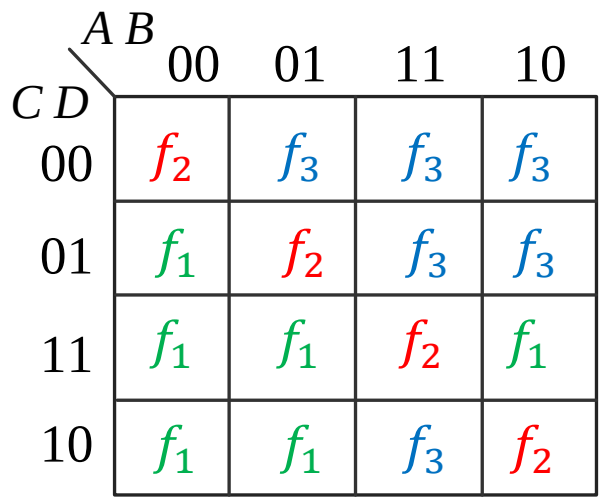

$$f_1 = \bar{A}C + \bar{A}\bar{B}D + \bar{B}CD$$




$$f_3 = A\bar{C} + B\bar{C}\bar{D} + AB\bar{D}$$



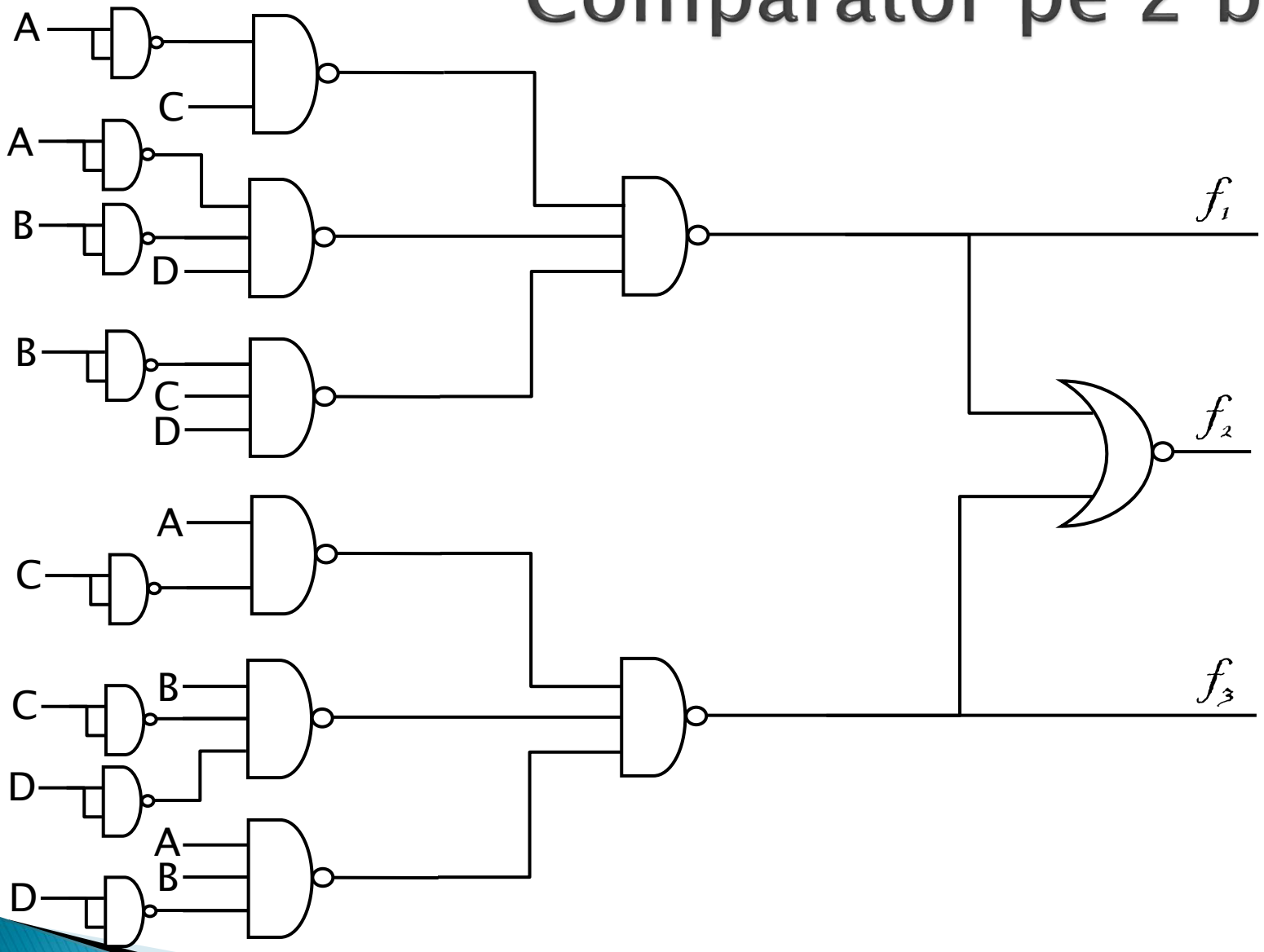

$$f_2 = \bar{A}\bar{B}\bar{C}\bar{D} + \bar{A}B\bar{C}D + ABCD + A\bar{B}C\bar{D}$$




$$f_1 + f_2 + f_3 = 1 \text{ (FNDC)}$$

$$f_2 = !(f_1 + f_3)$$

# Comparator pe 2 biți (2)



# Diagrame Karnaugh – Aplicația 2: convertor cod Gray → cod binar

- Realizați sinteza unui generator ce convertește semnalele din cod Gray în cod binar:




Convertor cod Gray → cod binar

- Cum arată circuitul care primește la intrare cod Gray și scoate la ieșire cod binar?

# Convertor cod Gray → cod binar (2)

- Pornim de la tabela de adevăr deoarece îmi permite să fac o analiză completă și rapidă.
- Câte variabile de intrare avem? 4
- Câte funcții de ieșire avem? 4
- De câte variabile de intrare depinde fiecare ieșire? 4: $G_3$ – $G_0$
- O astfel de problemă în care avem mai multe ieșiri, fiecare depinzând de toate variabilele de intrare se numește problemă cu ieșiri multiple.

# Convertor cod Gray → cod binar (3)

- Cod Gray → 2 linii consecutive diferă printr-un singur bit
- Calculăm cele 4 funcții de ieșire $B_0$ – $B_3$ (câte o diagramă pentru fiecare în parte)

| $G_3$ | $G_2$ | $G_1$ | $G_0$ | $B_3$ | $B_2$ | $B_1$ | $B_0$ |
|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | 0 | 0 | 0 | 1 |
| 0 | 0 | 1 | 1 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 0 | 0 | 0 | 1 | 1 |
| 0 | 1 | 1 | 0 | 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 1 | 0 | 1 | 0 | 1 |
| 0 | 1 | 0 | 1 | 0 | 1 | 1 | 0 |
| 0 | 1 | 0 | 0 | 0 | 1 | 1 | 1 |
| 1 | 1 | 0 | 0 | 1 | 0 | 0 | 0 |
| 1 | 1 | 0 | 1 | 1 | 0 | 0 | 1 |
| 1 | 1 | 1 | 1 | 1 | 0 | 1 | 0 |
| 1 | 1 | 1 | 0 | 1 | 0 | 1 | 1 |
| 1 | 0 | 1 | 0 | 1 | 1 | 0 | 0 |
| 1 | 0 | 1 | 1 | 1 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 | 1 | 1 | 1 | 0 |
| 1 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |

# Convertor cod Gray → cod binar (4)

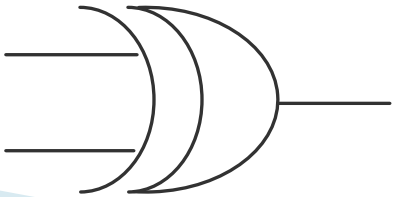
| $G_3G_2$ \ $G_1G_0$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | | | | |
| 01 | | | | |
| 11 | 1 | 1 | 1 | 1 |
| 10 | 1 | 1 | 1 | 1 |

$B_3 = G_3$

| $G_3G_2$ \ $G_1G_0$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | | | | |
| 01 | 1 | 1 | 1 | 1 |
| 11 | | | | |
| 10 | 1 | 1 | 1 | 1 |

$B_2 = G_3!G_2 + !G_3G_2$

Introducem operatorul logic XOR (SAU EXCLUSIV) $\oplus$
Având circuitul

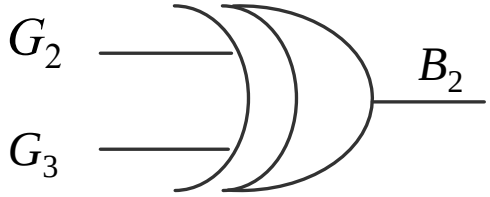


și tabela de adevăr:

| a | b | a ⊕ b |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

# Convertor cod Gray → cod binar (4)

- → $B_2 = G_3 \oplus G_2$ → implementarea este mult mai simplă cu XOR decât cu NOR/NAND:




| $G_3G_2$ \ $G_1G_0$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | | | 1 | 1 |
| 01 | 1 | 1 | | |
| 11 | | | 1 | 1 |
| 10 | 1 | 1 | | |

$B_1 = !G_3!G_2G_1 + !G_3G_2!G_1 + G_3G_2G_1 + G_3!G_2!G_1 = G_1(!G_3!G_2 + G_3G_2) + !G_1(!G_3G_2 + G_3!G_2)$

$!B_2 = !(G_3!G_2 + !G_3G_2) = (!G_3 + G_2)(!G_2 + G_3) = !G_3!G_2 + G_3G_2$

→ $B_1 = G_1!B_2 + !G_1B_2 = B_2 \oplus G_1$

# Convertor cod Gray → cod binar (5)

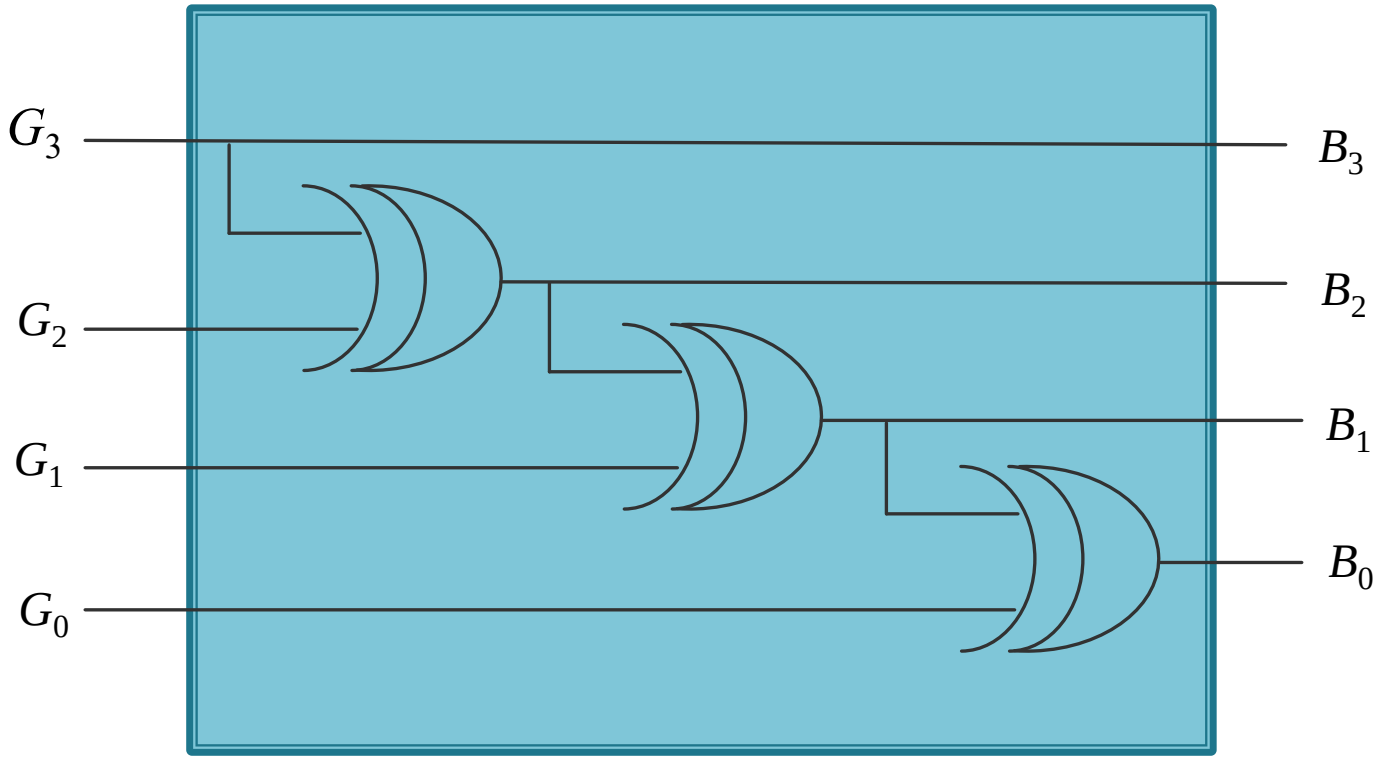
- $B_0 = G_0!B_1 + !G_0B_1 = G_0 \oplus B_1$
- $B_1 = G_1 \oplus B_2$
- $B_2 = G_2 \oplus G_3$
- $B_3 = G_3$

| $G_3G_2$ \ $G_1G_0$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | | 1 | | 1 |
| 01 | 1 | | 1 | |
| 11 | | 1 | | 1 |
| 10 | 1 | | 1 | |

$$B_0 = \overline{G_3}\ \overline{G_2}\ \overline{G_1}\ G_0 + \overline{G_3}\ \overline{G_2}\ G_1\overline{G_0} + \overline{G_3}\ G_2\overline{G_1}\ \overline{G_0} + G_3\overline{G_2}\ \overline{G_1}\ \overline{G_0} + \\ + \overline{G_3}G_2G_1G_0 + G_3\overline{G_2}G_1G_0 + G_3G_2\overline{G_1}G_0 + G_3G_2G_1\overline{G_0}$$

$$B_0 = G_0(\underbrace{\overline{G_3}\ \overline{G_2}\ \overline{G_1} + \overline{G_3}\ G_2G_1 + G_3\overline{G_2}\ G_1 + G_3G_2\overline{G_1}}_{\overline{B_1}}) + \\ + \overline{G_0}\ (\underbrace{G_3G_2G_1 + \overline{G_3}\ \overline{G_2}\ G_1 + \overline{G_3}\ G_2\overline{G_1} + G_3\overline{G_2}\ \overline{G_1}}_{B_1})$$

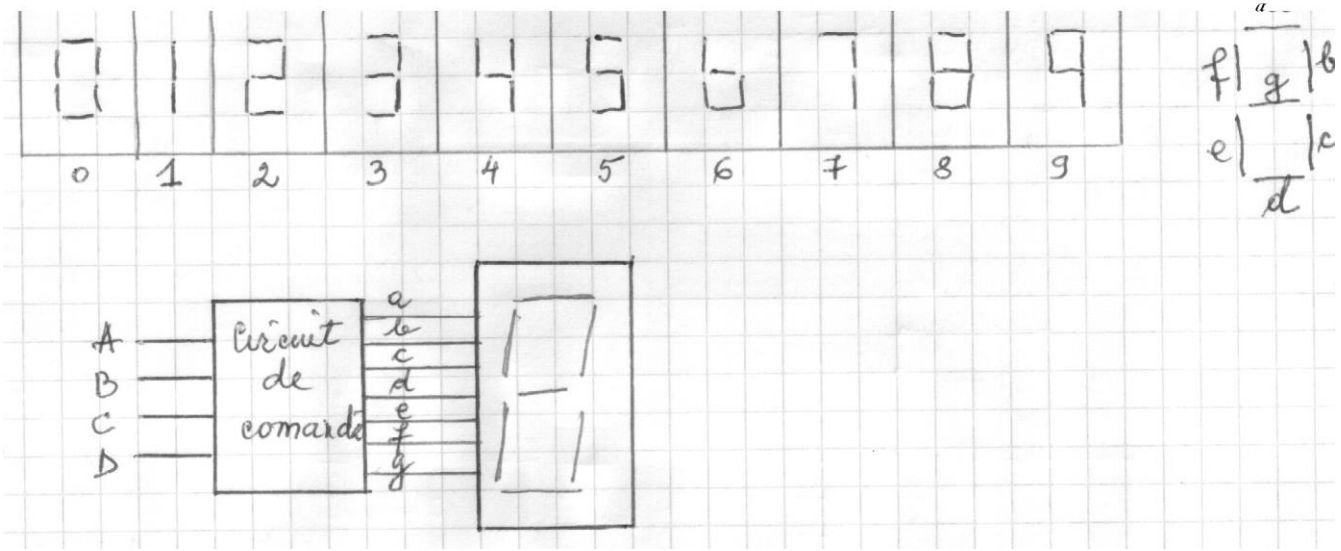
# Convertor cod Gray → cod binar (6)




Lungimea de cale a unui circuit e dată de numărul maxim de porți pe care le parcurge semnalul de la intrarea în circuit și până la ieșire. → Comparați implementarea cu XOR vs NOR/NAND din punctul de vedere al lungimii căii!

# MUX – Aplicația 3: Afișor cu 7 segmente

- Să se realizeze circuitul de comandă a dispozitivului de afișare numerică cu 7 segmente folosind MUX 4:1



# Tabelul de adevăr

| Cod zecimal | A | B | C | D | a | b | c | d | e | f | g |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 0 |
| 1 | 0 | 0 | 0 | 1 | 0 | 1 | 1 | 0 | 0 | 0 | 0 |
| 2 | 0 | 0 | 1 | 0 | 1 | 1 | 0 | 1 | 1 | 0 | 1 |
| 3 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 0 | 1 |
| 4 | 0 | 1 | 0 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 |
| 5 | 0 | 1 | 0 | 1 | 1 | 0 | 1 | 1 | 0 | 1 | 1 |
| 6 | 0 | 1 | 1 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 1 |
| 7 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 0 |
| 8 | 1 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 9 | 1 | 0 | 0 | 1 | 1 | 1 | 1 | 0 | 0 | 1 | 1 |
| 10 | 1 | 0 | 1 | 0 | * | * | * | * | * | * | * |
| .... | 1 | ... | ... | ... | * | * | * | * | * | * | * |

# Diagrame Karnaugh

| AB / CD | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | 1 | | * | 1 |
| 01 | | 1 | * | 1 |
| 11 | 1 | 1 | * | * |
| 10 | 1 | | * | * |

a

| AB / CD | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | 1 | 1 | * | 1 |
| 01 | 1 | | * | 1 |
| 11 | 1 | 1 | * | |
| 10 | 1 | | * | * |

b

| AB / CD | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | 1 | 1 | * | 1 |
| 01 | 1 | 1 | * | 1 |
| 11 | 1 | 1 | * | * |
| 10 | | 1 | * | * |

c

| AB / CD | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | 1 | | * | 1 |
| 01 | | 1 | * | |
| 11 | 1 | | * | * |
| 10 | 1 | 1 | * | * |

d

| AB / CD | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | 1 | | * | 1 |
| 01 | | | * | |
| 11 | | | * | * |
| 10 | 1 | 1 | * | * |

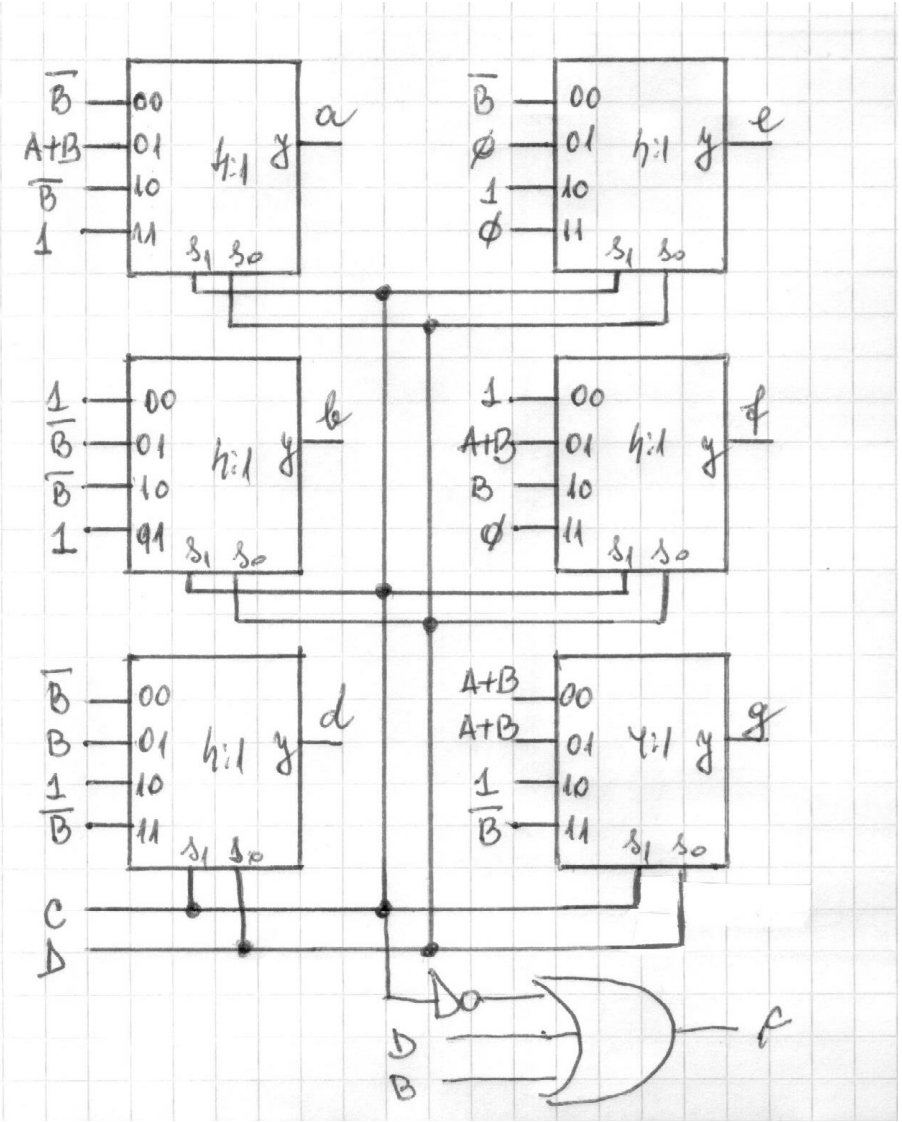
e

| AB / CD | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | 1 | 1 | * | 1 |
| 01 | | 1 | * | 1 |
| 11 | | | * | * |
| 10 | | 1 | * | * |

f

# Diagrame Karnaugh și implementare

| AB<br>CD | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | | 1 | * | 1 |
| 01 | | 1 | * | 1 |
| 11 | 1 | | * | * |
| 10 | 1 | 1 | * | * |
| | | g | | |



# ÎNTREBĂRI?